

# Università degli Studi di Torino

Dipartimento di informatica



**UNIVERSITÀ  
DI TORINO**

Tesi di Laurea Triennale in Informatica

## **Analisi comparativa di DuckDB, PostGIS e GeoPandas per l'elaborazione e l'analisi di dati geospaziali**

Relatore:

**Prof. Rossano Schifanella**

Candidato:

**Alessandro Demo**

Matricola 1049825



*You can't be afraid to fail. It's the only way you succeed - you're not gonna succeed all the time, and I know that.*

- LeBron James

## Dichiarazione di Originalità

*Dichiaro di essere responsabile del contenuto dell'elaborato che presento al fine del conseguimento del titolo, di non avere plagiato in tutto o in parte il lavoro prodotto da altri e di aver citato le fonti originali in modo congruente alle normative vigenti in materia di plagio e di diritto d'autore. Sono inoltre consapevole che nel caso la mia dichiarazione risultasse mendace, potrei incorrere nelle sanzioni previste dalla legge e la mia ammissione alla prova finale potrebbe essere negata.*

## Abstract

Gli strumenti attuali per l'analisi geospaziale presentano sia architetture consolidate, come PostGIS, che alternative moderne, come DuckDB e GeoPandas, richiedendo un'attenta valutazione dei loro trade-off.

Questa ricerca nasce per capire quale tecnologia utilizzare basandosi non solo sulle prestazioni, ma anche sull'affidabilità e sulla coerenza numerica dei risultati. A tal proposito, è stato sviluppato un framework di benchmarking, applicato a cinque casi d'uso reali su dataset di complessità crescente, per testare ogni tecnologia in diversi scenari operativi.

Dall'analisi comparativa emerge come le tecnologie si distinguano per punti di forza specifici, risultando più adatte a particolari casi d'uso e contesti applicativi. PostGIS eccelle nei calcoli geometrici complessi e GeoPandas nei join di prossimità, mentre DuckDB offre prestazioni elevate che dipendono da una precisa configurazione dei metadati.

Questo studio contribuisce ad una completa valutazione delle tecnologie geospaziali in analisi, concludendo che la scelta finale è un trade-off tra velocità, maturità dell'ecosistema e validazione dei dati, visto che le ricerche future potrebbero estendere l'analisi a nuovi strumenti, dataset più estesi e casi d'uso più complessi.



# Indice

|          |                                                               |           |
|----------|---------------------------------------------------------------|-----------|
| <b>1</b> | <b>Introduzione</b>                                           | <b>5</b>  |
| 1.1      | Contesto . . . . .                                            | 5         |
| 1.1.1    | Il riferimento tradizionale: i database spaziali . . . . .    | 5         |
| 1.1.2    | La nuova ondata: analisi in-process e in-memory . . . . .     | 5         |
| 1.1.3    | Il banco di prova: OpenStreetMap . . . . .                    | 6         |
| 1.2      | Obiettivi della tesi . . . . .                                | 6         |
| 1.3      | Struttura per lo sviluppo del progetto . . . . .              | 7         |
| 1.4      | Struttura organizzativa del progetto . . . . .                | 9         |
| <b>2</b> | <b>I dati geospaziali</b>                                     | <b>13</b> |
| 2.1      | Che cosa sono i dati geospaziali . . . . .                    | 13        |
| 2.2      | I modelli di dati geospaziali analizzati . . . . .            | 13        |
| 2.2.1    | OpenStreetMap . . . . .                                       | 14        |
| 2.2.2    | Vettoriali puri . . . . .                                     | 15        |
| 2.2.3    | Raster . . . . .                                              | 15        |
| 2.2.4    | I dataset nello specifico . . . . .                           | 16        |
| <b>3</b> | <b>DuckDB, il database in-process con estensione spaziale</b> | <b>19</b> |
| 3.1      | Che cos'è DuckDB? . . . . .                                   | 19        |
| 3.2      | Caratteristiche principali . . . . .                          | 19        |
| 3.3      | L'estensione Spatial . . . . .                                | 21        |
| 3.4      | Ruolo nel progetto . . . . .                                  | 22        |
| <b>4</b> | <b>PostGIS, l'estensione spaziale di PostgreSQL</b>           | <b>23</b> |
| 4.1      | Che cos'è PostGIS? . . . . .                                  | 23        |
| 4.2      | Architettura e caratteristiche principali . . . . .           | 23        |
| 4.3      | Ruolo nel progetto . . . . .                                  | 25        |
| <b>5</b> | <b>GeoPandas, la libreria Python per dati geospaziali</b>     | <b>27</b> |
| 5.1      | Che cos'è GeoPandas? . . . . .                                | 27        |
| 5.2      | L'ecosistema GeoPandas . . . . .                              | 28        |
| 5.3      | Ruolo nel progetto . . . . .                                  | 29        |
| <b>6</b> | <b>I casi d'uso, cinque scenari di benchmark</b>              | <b>31</b> |
| 6.1      | Casi d'uso per dati vettoriali OSM . . . . .                  | 31        |
| 6.1.1    | Caso d'uso 1: Ingestione Dati . . . . .                       | 31        |
| 6.1.2    | Caso d'uso 2: Filtraggio ed Estrazione Dati . . . . .         | 32        |
| 6.1.3    | Caso d'uso 3: Analisi su Singola Tabella . . . . .            | 32        |
| 6.1.4    | Caso d'uso 4: Join Spaziale Complesso . . . . .               | 33        |

|          |                                                                  |           |
|----------|------------------------------------------------------------------|-----------|
| 6.2      | Casi d'uso per dati vettoriali puri e raster . . . . .           | 34        |
| 6.2.1    | Caso d'uso 1: Ingestione Dati . . . . .                          | 34        |
| 6.2.2    | Caso d'uso 2: Filtraggio ed Estrazione Dati . . . . .            | 34        |
| 6.2.3    | Caso d'uso 5: Analisi Vettoriale-Raster . . . . .                | 35        |
| <b>7</b> | <b>I risultati ottenuti</b>                                      | <b>37</b> |
| 7.1      | I risultati nello specifico . . . . .                            | 37        |
| 7.1.1    | Casi d'uso 1 e 2: Ingestione e Filtraggio Dati vettoriali OSM .  | 38        |
| 7.1.2    | Casi d'uso 1 e 2: Ingestione e Filtraggio Dati vettoriali puri . | 39        |
| 7.1.3    | Casi d'uso 1 e 2: Ingestione e Filtraggio Dati raster . . . . .  | 40        |
| 7.1.4    | Caso d'uso 3: Analisi su Singola Tabella . . . . .               | 41        |
| 7.1.5    | Caso d'uso 4: Join Spaziale Complesso . . . . .                  | 44        |
| 7.1.6    | Caso d'uso 5: Analisi Vettoriale-Raster . . . . .                | 47        |
| 7.2      | Consigli applicativi . . . . .                                   | 47        |
| 7.3      | Scoperte significative . . . . .                                 | 49        |
| <b>8</b> | <b>Conclusioni</b>                                               | <b>53</b> |
| 8.1      | Limitazioni dello studio . . . . .                               | 53        |
| 8.2      | Sviluppi futuri . . . . .                                        | 54        |
|          | <b>Ringraziamenti</b>                                            | <b>58</b> |
|          | <b>Bibliografia</b>                                              | <b>67</b> |

# 1. Introduzione

Il ruolo dei dati geospaziali è diventato pervasivo, influenzando decisioni critiche in ambiti che spaziano dalla pianificazione urbana alla logistica e alla gestione ambientale. Di pari passo con questa crescente centralità, l'ecosistema software si è evoluto, ponendo gli analisti e gli sviluppatori davanti ad una scelta. Da un lato persistono le architetture di database consolidate, da tempo garanzia di robustezza; dall'altro emergono nuovi strumenti di analisi, più agili e veloci, incentrati sull'elaborazione diretta di file.

Questa tesi si inserisce in questo scenario di innovazione tecnologica, con l'intento di fornire un'analisi completa per affrontare questa scelta. L'obiettivo è andare oltre una semplice misurazione delle tempistiche di esecuzione, esplorando ogni possibile trade-off tra gli approcci tradizionali e quelli moderni in termini di prestazioni, robustezza, coerenza numerica e, soprattutto, praticità d'uso.

## 1.1. Contesto

La necessità di effettuare questo studio nasce dalla convergenza di tre tendenze principali nel mondo dei dati geospaziali: la crescita dei database spaziali tradizionali, l'emergere di nuovi motori analitici in-process e la disponibilità di dati open source e complessi come quelli di OpenStreetMap.

### 1.1.1. Il riferimento tradizionale: i database spaziali

Per decenni, il modello di riferimento per la gestione di grandi quantità di dati geospaziali è stato il database relazionale esteso con funzionalità spaziali. In questo campo, **PostgreSQL** [63], con la sua estensione **PostGIS** [58], rappresenta lo standard open source. Questo approccio centralizzato, basato su un'architettura client-server, offre enormi vantaggi in termini di integrità dei dati, gestione di accessi concorrenti e sicurezza, oltre a una vasta gamma di funzionalità geometriche e analitiche sviluppate e migliorate in più di vent'anni. Tuttavia, questa tecnologia comporta una maggiore complessità iniziale in termini di installazione, configurazione e ingestione dei dati, fasi che richiedono lo sviluppo di procedure ben definite e l'uso di strumenti specifici.

### 1.1.2. La nuova ondata: analisi in-process e in-memory

Il mondo della data science ha visto una rapida e continua affermazione di strumenti sempre più veloci, interattivi e semplici, spostando l'elaborazione dai server esterni

direttamente all'interno degli script di analisi. Tra questi strumenti, due hanno richiamato una particolare attenzione:

- **DuckDB**, un innovativo database analitico **in-process** [40]. Fu progettato per essere l'equivalente di SQLite [70] per l'analisi di dati, offrendo la potenza di un motore SQL colonnare e vettorizzato senza la necessità della presenza di un sistema di gestione di database (DBMS) [44]. La sua estensione **Spatial** [45] permette di eseguire query complesse direttamente su file geospaziali, come i *GeoParquet*, con una configurazione minima ma a prestazioni elevate.
- **GeoPandas**, una libreria che ha reso l'analisi geospaziale accessibile nell'ecosistema Python per la data science [33]. Sfruttando la potenza delle librerie su cui si basa, come *pandas* per i dati tabulati, *Shapely* e *GEOS* per le operazioni geometriche, *GeoPandas* permette di eseguire analisi complesse interamente **in-memory** [36]. Questo approccio è flessibile e potente, ma potenzialmente limitato dalla RAM disponibile all'utente.

### 1.1.3. Il banco di prova: OpenStreetMap

La disponibilità di dati *vettoriali* open source dettagliati a livello globale, come quelli provenienti da **OpenStreetMap (OSM)** [20], fornisce il perfetto banco di prova per effettuare confronti tra queste tipologie di sistemi. *OSM* non è un dataset pulito e strutturato, bensì un progetto collaborativo nato per mappare l'intero globo, i cui precisi dettagli sono accompagnati da una notevole complessità strutturale (*node*, *way* e *relation*) [18], portandoli anche a essere di dimensioni considerevoli [17].

Tutte le tecnologie che elaborano dati di *OSM* devono poter gestire non solo la scala, ma anche la “disordinata” natura dei dati del mondo reale, mettendo a dura prova sia la velocità che la robustezza di ogni algoritmo. Proprio per questo motivo è nata la domanda di ricerca di questa tesi: in scenari operativi concreti e con dati reali, i nuovi strumenti possono eguagliare o, addirittura, superare l'affidabilità e la potenza dei sistemi tradizionali?

## 1.2. Obiettivi della tesi

Questo progetto di tesi è stato sviluppato per condurre un'analisi comparativa sulle prestazioni delle diverse tecnologie geospaziali, sia moderne che tradizionali, applicate a operazioni che variano dall'ingestione all'analisi di dati su larga scala.

I dati su cui si basa la tesi sono principalmente quelli provenienti da *OpenStreetMap*, ma l'analisi è estesa anche a dati **vettoriali puri** [83] e **raster** [81], così da effettuare una valutazione più completa.

La ricerca nasce dall'attuale diffusione crescente di strumenti per l'analisi di dati scientifici e di database analitici *in-process*, portando a domandarsi quali siano i **trade-off** rispetto all'utilizzo di tecnologie consolidate da tempo, come i database server relazionali.

L'obiettivo finale è dunque misurare qualitativamente le prestazioni di sistemi **file-based** (*DuckDB* con estensione *Spatial* e *GeoPandas*) con il rendimento di sistemi basati direttamente su database esterni (*PostgreSQL* con estensione spaziale *PostGIS*), in termini di velocità, consumo delle risorse, complessità implementativa e coerenza dei risultati.

Per effettuare queste misure, sono stati ideati cinque diversi **casi d'uso** [99] realistici, che trattano dall'ingestione diretta dei dati fino a veri e propri **join** [27] complessi.

### 1.3. Struttura per lo sviluppo del progetto

Il progetto adotta un framework basato su tre componenti fondamentali: **Technologies**, **Data** e **Libraries**, da qui l'acronimo **TDL**. Sfruttando questo approccio, si possono distinguere in maniera chiara i *motori* di elaborazione dei benchmark, i *dati* in ingresso su cui operano i motori e gli *strumenti software* che permettono l'ingestione dei dati.

#### 1. *Technologies (T)*, i motori di elaborazione

Le tecnologie rappresentano tutti i sistemi sottoposti ai benchmark, ossia la base per l'esecuzione delle operazioni geospaziali in analisi.

In particolare, si vanno ad analizzare le tre tecnologie seguenti:

- *DuckDB* con estensione *Spatial*, un moderno database SQL *in-process*. È veloce, leggero e ottimizzato per letture su formati colonnari come *GeoParquet* (è una estensione del formato *Apache Parquet* per immagazzinare dati geospaziali [29]).
- *PostgreSQL* con estensione *PostGIS*, un sistema di gestione di database relazionale che è ormai da anni il sistema di riferimento industriale. È noto per essere robusto, sempre aggiornato e disporre di funzionalità spaziali estremamente complete.
- *GeoPandas*, che combina le funzionalità di altre librerie come *pandas* e *Shapely*, è l'approccio in "Python puro". Rappresenta l'analisi *in-memory*, visto che l'intero dataset viene caricato e manipolato direttamente in RAM.

#### 2. *Data (D)*, i dati di input

I dati definiscono i formati, le scale e le tipologie di input utilizzati per testare le tecnologie.

I modelli di dati che vengono utilizzati sono:

- *Vettoriali OSM*, dati estratti dalla mappatura di *OpenStreetMap*, che possono essere di diverse dimensioni per valutare al meglio la scalabilità delle tecnologie in analisi.

I dati di *OSM* sfruttano formati come:

- **Protocolbuffer Binary Format (PBF)** [21], compatto e flessibile.
- **Geographic JSON (GeoJSON)** [13], utilizzato per gestire collezioni di geometrie spaziali.
- **GeoParquet** [14], permette di salvare i dati in formato colonnare.
- *Vettoriali puri*, dati non derivati da *OSM*, come i confini amministrativi ISTAT in formato **Shapefile** [30], per testare le prestazioni su formati *vettoriali* standard.

- *Raster*, dati capaci di costruire immagini digitali composte da pixel. I dati *raster* più comuni sono i *Digital Elevation Model (DEM)* [101] in formato **Georeferenced Tagged Image File Format (GeoTIFF)** [39], usati per valutare le capacità di analisi su dati a griglia.

### 3. *Libraries (L)*, gli strumenti di ingestione.

Le librerie costituiscono lo strato software che si occupa di leggere i dati e di caricarli per la tecnologia in uso. Per fare ciò, si sfrutta il processo **Extract, Transform and Load (ETL)** [4].

Le librerie utilizzate per eseguire correttamente tutti i benchmark del progetto sono le seguenti:

- **quackosm** [75], una libreria Python che permette l'interfacciamento diretto dei file *PBF* con *DuckDB*.
- **osm2pgsql** [73], uno strumento a riga di comando che rende possibile l'importazione massiccia di dati provenienti da *OSM* in un database *PostGIS*.
- **shp2pgsql** [61], uno strumento a riga di comando, incluso con l'installazione di *PostGIS*, che converte i file *Shapefile* per l'inserimento dei dati direttamente in una tabella. È il metodo standard per l'ingestione di dati *vettoriali puri*.
- **raster2pgsql** [59], uno strumento a riga di comando analogo a *shp2pgsql*, specifico per l'ingestione di dati *raster* in *PostGIS*. Converte il file *raster* in un formato SQL che può essere caricato come una tabella di tasselli (tiles) *raster* nel database.
- **pyosmium** [65], una libreria Python di basso livello per effettuare parsing di file *PBF*. Viene utilizzata per popolare i *GeoDataFrame* di *GeoPandas*.
- **osmium-tool** [91], uno strumento a riga di comando che viene usato per operazioni di pre-processing sui file *PBF*.
- **rasterio** [49], la libreria Python di riferimento per la manipolazione di dati geospaziali *raster*. Fornisce un'interfaccia per leggere, scrivere e processare file in formato *GeoTIFF*.
- **rioxarray** [78], una libreria Python che estende altre librerie, come *xarray* e *rasterio*, per fornire un'interfaccia di alto livello per l'analisi di dati *raster*.
- **geoalchemy2** [5], una libreria Python che fa da intermediario tra *GeoPandas* e il database *PostGIS*, grazie al suo tool *SQLAlchemy* [2], permettendo di scrivere i *GeoDataFrame* direttamente in tabelle spaziali su *PostgreSQL*.
- **pandas** [79], una libreria Python fondamentale per effettuare manipolazione di dati. Fornisce la struttura *DataFrame* su cui si basa *GeoPandas* e viene usata per gestire i risultati tabellari non geografici.
- **osmnx** [6], un package Python realizzato usando *OpenStreetMap* che semplifica il download di dati spaziali, fornendo geometrie di base affidabili per poterci eseguire operazioni sopra.

## 1.4. Struttura organizzativa del progetto

Il progetto è organizzato in una struttura modulare per garantire chiarezza e riproducibilità:

- /DuckDB:
  - `duckdb_scalability_test.ipynb`: file Jupyter Notebook in cui sono presenti tutti i test di benchmarking e scalabilità effettuati su *DuckDB* con l'estensione *Spatial*.
  - `test_number1.ipynb`: file Jupyter Notebook in cui sono stati effettuati esempi su come utilizzare *DuckDB*, sia con che senza l'estensione *Spatial*.
- /GeoPandas:
  - `test_number1.ipynb`: file Jupyter Notebook in cui sono stati effettuati esempi su come utilizzare la libreria Python *GeoPandas*.
- /OSMnx:
  - `test_number1.ipynb`: file Jupyter Notebook in cui sono stati effettuati esempi su come utilizzare il package Python *osmnx*.
- /QuackOSM:
  - `test_number1.ipynb`: file Jupyter Notebook in cui sono stati effettuati esempi su come utilizzare la libreria Python *quackosm*.
- /Rasterio:
  - `test_number1.ipynb`: file Jupyter Notebook in cui sono stati effettuati esempi su come utilizzare la libreria Python *rasterio*.
- /UseCasesManagement:
  - `/instructions`: contiene i file di testo (`.txt`) che spiegano come effettuare comandi complessi per svolgere al meglio le analisi e i confronti.
    - \* `postgis_commands.txt`: file di testo usato come promemoria per conservare i comandi lunghi e complessi da usare via terminale per gli strumenti esterni *osm2pgsql*, *shp2pgsql* e *raster2pgsql*, facilitandone il riutilizzo senza errori.
    - \* `pyosmium_commands.txt`: file di testo usato come promemoria per conservare i comandi da usare via terminale per lo strumento esterno *osmium extract*, in modo tale da poterli riutilizzare ed evitando di commettere errori.
  - `/notebooks`: contiene i Jupyter Notebook (`.ipynb`) usati per l'esplorazione dei dati, il debugging, la generazione di file *GeoJSON* su discrezione dell'utente e la visualizzazione dei risultati ottenuti dalle analisi.
    - \* `get_boundaries.ipynb`: file Jupyter Notebook per scaricare, facendo uso della libreria *osmnx*, il poligono di confine ufficiale di una determinata area e salvarlo come file *GeoJSON*. È stato creato per effettuare il benchmark del caso d'uso 1 usando la libreria *osmium-tool*.

- \* `ingesting_and_filtering_data.ipynb`: file Jupyter Notebook usato per la fase di esplorazione e debugging. Avviene principalmente la conversione di file *PBF* in formato *GeoParquet* sfruttando la libreria *quackosm*.
- `/results`: contiene gli output quantitativi dei benchmark.
  - \* `benchmark_results.csv`: file CSV che funziona da database centrale per tutti i risultati quantitativi delle analisi di progetto. Ogni riga rappresenta un singolo test di benchmark e viene aggiunta automaticamente dagli script al termine di un'esecuzione, garantendo una raccolta dati strutturata e coerente.
- `/scripts`: contiene gli script Python (`.py`) utilizzati per eseguire i benchmark automatizzati e riproducibili per ogni *caso d'uso*. Ogni script dedicato a un *caso d'uso* farà 100 esecuzioni, la prima cold start (a freddo) e le altre 99 in hot cache (a caldo, visto che i dati sono già stati “visti” dal computer).
  - \* `/uc1_and_uc2`: contiene gli script usati per eseguire i benchmark dei casi d'uso 1 e 2.
    - `uc1_and_uc2_duckdb_benchmark.py`: script di benchmark per i casi d'uso 1 e 2 che testa la tecnologia *DuckDB* combinata con la libreria *quackosm*. Serve a dirigere l'intero processo di ingestione, filtrando un grande file *PBF* per una specifica area geografica e per tag (es.: `building=true`), salvando l'output in formato *GeoParquet*. Misura e registra le performance di questo approccio più moderno.
    - `uc1_and_uc2_postgis_benchmark.py`: script di benchmark per i casi d'uso 1 e 2 relativo alla tecnologia *PostGIS*. Questo script non esegue l'importazione (che viene fatta a parte con lo strumento *osm2pgsql* su *PostgreSQL*), ma si connette al database già popolato per misurare esclusivamente la velocità di una query di estrazione spaziale (Query Time). Dimostra la performance di interrogazione di un database tradizionale.
    - `uc1_and_uc2_pyosmium_benchmark.py`: script di benchmark per i casi d'uso 1 e 2 che implementa l'approccio Python puro con la tecnologia *GeoPandas* combinata con la libreria *pyosmium*. Legge un file *PBF*, ricostruisce le geometrie degli edifici in memoria e le carica in un *GeoDataFrame*, che viene poi filtrato per l'area di interesse. Valuta le performance di un metodo che non dipende da database o strumenti esterni.
    - `uc1_and_uc2_raster_data_benchmark.py`: script di benchmark per i casi d'uso 1 e 2 eseguiti esclusivamente su dati *raster*, testando le capacità di ingestione e filtraggio di *PostGIS*, tramite lo strumento *raster2pgsql*, e di Python, usando la libreria *rasterio*.
    - `uc1_and_uc2_vector_data_benchmark.py`: script di benchmark per i casi d'uso 1 e 2 eseguiti solo su dati *vettoriali puri*, confrontando le performance di tutte e tre le tecnologie in analisi su un formato standard. *PostGIS* sfrutta esternamente lo strumento *shp2pgsql* per importare i dati nel database.

- \* /uc3: contiene gli script usati per eseguire i benchmark del caso d'uso 3.
  - `uc3_duckdb_benchmark.py`: script di benchmark per il caso d'uso 3 che testa le performance di *DuckDB* con estensione *Spatial*. Esegue 3 diverse query analitiche su 9 differenti file *GeoParquet* già processati. Include la logica di trasformazione delle coordinate *CRS* per garantire la correttezza dei calcoli metrici e salva i risultati nel file CSV.
  - `uc3_postgis_benchmark.py`: script di benchmark per il caso d'uso 3 che traduce le operazioni analitiche di *DuckDB* in query SQL specifiche per *PostGIS*. Esegue le stesse 3 query sui 9 dataset precedentemente caricati nel database. Include anch'esso la logica di trasformazione delle coordinate *SRID* per garantire la correttezza dei calcoli metrici e salva i risultati nel file apposito.
  - `uc3_geopandas_benchmark.py`: script di benchmark per il caso d'uso 3 che implementa le medesime 3 operazioni, ma utilizzando lo stack Python nativo con *GeoPandas*. Esegue i test sempre sugli stessi 9 dataset leggendo direttamente da essi i dati e viene effettuata la trasformazione delle coordinate *CRS* per continuare a garantire la correttezza dei calcoli metrici.
- \* /uc4: contiene gli script usati per eseguire i benchmark del caso d'uso 4.
  - `uc4_duckdb_benchmark.py`: script di benchmark per il caso d'uso 4 che esegue 3 operazioni di join spaziale complesso (Point-In-Polygon, join di prossimità a catena e differenza geometrica) per valutare le performance di *DuckDB* su query analitiche avanzate.
  - `uc4_postgis_benchmark.py`: script di benchmark per il caso d'uso 4 che esegue 3 query per le operazioni di join spaziale complesso su tabelle *PostGIS*, ottenute sfruttando il file `geoparquet_loading_postgis.py`.
  - `uc4_geopandas_benchmark.py`: script di benchmark per il caso d'uso 4 che implementa le stesse 3 operazioni di join spaziale complesso, eseguite in precedenza usando *DuckDB* e *PostGIS* come database, utilizzando, invece, lo stack Python nativo con *GeoPandas*.
- \* /uc5: contiene gli script usati per eseguire i benchmark del caso d'uso 5.
  - `uc5_vector_raster_analysis_benchmark.py`: script di benchmark per il caso d'uso 5 che esegue l'analisi delle statistiche zonali per calcolare la popolazione in ogni comune della regione selezionata, confrontando le prestazioni di *PostGIS* e dello stack Python, che fa uso nello specifico della libreria *rasterstats* [74]. La query di *PostGIS* opera su una tabella pre-processata e pulita per garantire l'affidabilità dei risultati.
- \* `debug_quackosm.py`: script Python creato esclusivamente per scopi di diagnostica e debugging durante la risoluzione di problemi complessi con le librerie *quackosm* e *osmium-tool*.

- \* `debug_rasterio.py`: script Python creato esclusivamente per scopi di diagnostica durante la risoluzione di problemi complessi con la libreria *rasterio*, soprattutto per l'ingestione di dati via *PostgreSQL*.
- \* `geometries_fixing_and_gpkg_data_loading.py`: script Python di utilità *ETL* (*Extract, Transform and Load*) creato per risolvere i problemi di validità delle geometrie nel dataset dei comuni ISTAT. Esegue la lettura dello *Shapefile*, corregge le geometrie, le riproietta sotto *CRS* del *raster* *GHS-POP* e, infine, carica il dato pulito in una nuova tabella nel database di *PostGIS*, rendendola pronta all'uso per l'analisi del caso d'uso 5.
- \* `geoparquet_loading_postgis.py`: script Python di utilità che si occupa di caricare tutti i dati dei 24 file *GeoParquet* (edifici, fermate dei bus, ospedali, quartieri, parchi, strade residenziali, ristoranti e alberi di Pinerolo, Milano e Roma) nel database *PostGIS*, creando le tabelle necessarie per effettuare l'esecuzione del benchmark per il caso d'uso 3. Inoltre, misura anche il tempo di ingestione per ogni tabella.
- \* `benchmark_utils.py`: script Python di utilità centrale per il progetto. Contiene la classe `Timer` per una misurazione precisa dei tempi di esecuzione e la funzione `save_results`, che gestisce in modo robusto la scrittura dei risultati nel file `benchmark_results.csv`, creando automaticamente la cartella e le intestazioni, se necessario. Viene importato da tutti gli altri script di benchmark per garantirne la coerenza.

## 2. I dati geospaziali

La scelta, la comprensione e la gestione dei dati sono attività fondamentali in qualsiasi progetto di analisi geospaziale.

Questo capitolo della tesi descrive la natura dei **dati geospaziali** e i **modelli** fondamentali con cui vengono rappresentati, analizzando in dettaglio **le tipologie di dati e i dataset specifici utilizzati** come "banco di prova" per i benchmark eseguiti in questo progetto.

### 2.1. Che cosa sono i dati geospaziali

Un dato geospaziale [26], o geografico, è un'informazione che descrive oggetti, eventi o fenomeni che possiedono una posizione specifica sulla superficie terrestre e sono legati ad ulteriori informazioni. Queste informazioni si dividono in:

- Informazioni sugli attributi, come le caratteristiche dell'oggetto in analisi.
- Informazioni temporali, ossia la durata in cui la posizione e gli attributi dell'oggetto esistono.

Nello specifico, un dato geospaziale può essere un'immagine satellitare, un dato presente sui social media, un dato meteorologico o perfino un dato appartenente a tecnologie d'uso quotidiano come i telefoni cellulari.

Ciò che differenzia tutti i dati geospaziali da qualsiasi altro tipo di dato è la loro componente spaziale, che viene misurata tramite un sistema di coordinate specifico, il **Coordinate Reference System (CRS)** [108].

Inoltre, i dati geospaziali, per comporre rappresentazioni cartografiche, vengono gestiti tramite il **Geographic Information System (GIS)** [28], un sistema utile per analizzare e visualizzare informazioni geografiche [77].

### 2.2. I modelli di dati geospaziali analizzati

La rappresentazione digitale di queste informazioni si basa su due modelli concettuali che cercano di astrarre le complessità del mondo reale: il *vettoriale* e il *raster*.

In questo progetto, si analizzano i dati provenienti da tre diverse fonti e modelli rappresentativi:

- Dati *vettoriali* *OpenStreetMap (OSM)*: rappresentano un modello ibrido generato dagli utenti del progetto medesimo, basato su primitive vettoriali ma con una propria e approfondita struttura semantica e topologica.

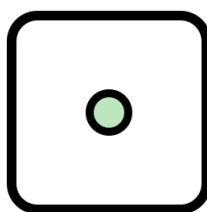
- Dati *vettoriali puri*: il modello classico per rappresentare informazioni discrete con confini precisi.
- Dati *raster*: il modello di riferimento per descrivere fenomeni continui su una griglia regolare, costituita da pixel [103].

### 2.2.1. OpenStreetMap

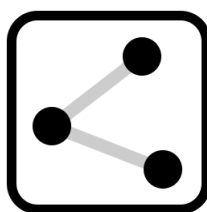
*OpenStreetMap* (*OSM*) è un progetto open source a livello globale nato per creare una mappa del mondo completamente libera e modificabile [20]. Spesso definito "Wikipedia delle mappe", il suo database è costruito e mantenuto da una comunità di utenti volontari, grazie ai quali costituisce una delle fonti più ricche e dettagliate di dati geospaziali a livello mondiale.

Il modello dati di *OSM* è topologico e si basa su tre elementi fondamentali:

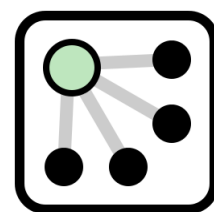
- **Node (Nodo)** [19]: un singolo punto nello spazio, definito da latitudine, longitudine e un identificatore univoco. Rappresenta elementi puntuali come alberi, fermate dell'autobus o vertici di geometrie più complesse.
- **Way (Percorso)** [24]: una lista ordinata di nodi che descrive un elemento lineare, come un fiume, o poligonale, come un edificio. Se un percorso è chiuso (ossia che il primo e l'ultimo nodo coincidono) e possiede almeno uno specifico *tag* (etichetta), viene interpretato come un'area.
- **Relation (Relazione)** [22]: un elemento che raggruppa altri elementi (nodi, vie e altre relazioni) per definire geometrie complesse, come multipoligoni (es.: un edificio con un cortile interno) o relazioni logiche (es.: un percorso di un autobus che include più strade).



(a) Nodo [19]



(b) Percorso [24]



(c) Relazione [22]

**Figura 2.2.1:** Esempi grafici dei tre elementi basilari del modello dati di *OpenStreetMap*.

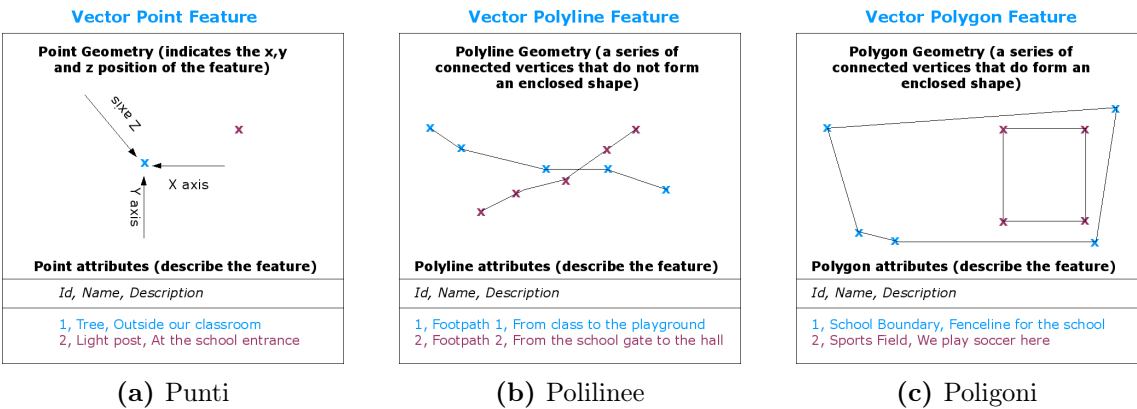
L'informazione semantica in *OSM* è affidata a un sistema di *tag*, ovvero etichette con coppie 'chiave=valore' associate a ogni elemento [23]. Ad esempio, un percorso può avere il tag 'highway=residential' per indicare una strada residenziale, oppure un poligono può essere etichettato come 'leisure=park' per definirlo come un parco. Questo sistema di *tag* viene utilizzato nel progetto per filtrare ed estrarre i dataset specifici.

Il formato di distribuzione più comune per i dati di *OSM* è il *Protocolbuffer Binary Format* (.pbf), una rappresentazione di dati binaria compatta e ottimizzata.

2.2.2. Vettoriali puri

Il modello dati *vettoriale* rappresenta gli oggetti del mondo reale utilizzando geometrie discrete con coordinate specifiche [83]. Questo modello è ideale per rappresentare oggetti con confini ben definiti, come strade, confini amministrativi oppure edifici. Le geometrie vettoriali basilari sono:

- **Punti** [86]: geometrie zero-dimensionali, definite da una singola coppia di coordinate (X, Y) ed occasionalmente da una terza coordinata (Z). Vengono usate per rappresentare elementi la cui estensione è trascurabile rispetto alla scala della mappa, come un ristorante o un albero.
- **Polilinee** [85]: geometrie monodimensionali, definite da una sequenza di vertici connessi. Rappresentano elementi lineari che hanno una lunghezza ma non un'area significativa, come una strada o un fiume.
- **Poligoni** [84]: geometrie bidimensionali, definite da una sequenza di vertici connessi da linee continue che racchiudono un'area, come ad esempio un edificio o un comune. In dataset topologici, i poligoni adiacenti condividono delle loro geometrie, spesso linee di confine.



**Figura 2.2.2:** Esempi grafici delle tre primitive fondamentali del modello dati *vettoriale* [83].

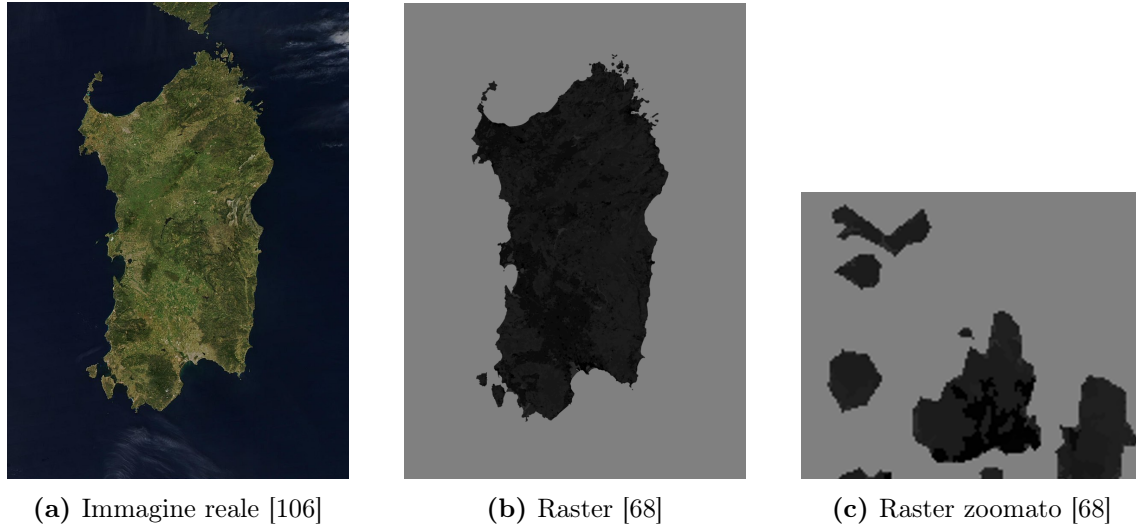
Ad ogni oggetto *vettoriale* è associata una serie di attributi che ne descrivono le caratteristiche (es.: il nome o il tipo), solitamente memorizzati in una tabella collegata [80]. In questo progetto, il formato utilizzato per i dati *vettoriali puri* è lo *Shapefile* (.shp), impiegato per analizzare le prestazioni delle tecnologie su dati non provenienti da *OSM*.

2.2.3. Raster

I dati *raster* rappresentano il territorio con una griglia (o matrice) di celle di uguali dimensioni, comunemente chiamate pixel [81]. A differenza del modello *vettoriale*, che descrive oggetti discreti, il modello *raster* è ideale per rappresentare fenomeni continui, dove ogni cella della griglia contiene un valore numerico che rappresenta una misurazione per quella specifica porzione di superficie. Esempi tipici sono l'altitudine, la temperatura o, nel caso di questo progetto, la densità di popolazione.

La caratteristica principale di un dato *raster* è la sua *risoluzione spaziale* [82], che definisce la dimensione a terra di una singola cella (es.: 100 metri x 100 metri). Le celle più piccole hanno una risoluzione più alta che implica un maggior dettaglio, ma anche una dimensione del file maggiore.

Per questo progetto è stato utilizzato il formato *GeoTIFF* (.tif), uno standard per le immagini *raster* georeferenziate, in modo tale da poter testare le capacità di analisi *vettoriale-raster*, in particolare con il dataset GHS-POP di Copernicus [7].



**Figura 2.2.3:** Esempi per confrontare immagini satellitari, *raster* non zoomati e *raster* zoomati in formato *GeoTIFF*.

#### 2.2.4. I dataset nello specifico

I benchmark di questo progetto vengono eseguiti utilizzando diversi dataset per testare le tre tecnologie su formati e scale differenti. I dataset in questione sono stati suddivisi in due tipologie precise:

- **Dataset di origine:** questi file costituiscono i dataset di partenza, acquisiti dalle rispettive fonti ufficiali e utilizzati come input per le operazioni di elaborazione e derivazione dei dataset sperimentali.
  - `italy-latest.osm.pbf` [46]: dataset *OSM* in formato *PBF* contenente l'intera nazione italiana. È stato scaricato da Geofabrik [47] e utilizzato come fonte primaria per le operazioni di estrazione su aree specifiche, sfruttando strumenti basati su indici.
  - `lombardy-latest.osm.pbf`: dataset *OSM* regionale, creato tramite un'estrazione dal file nazionale con *osmium-tool*. È stato generato per rendere più efficienti i test su tecnologie che necessitano di caricare l'intero file in memoria, come lo stack *GeoPandas* con *pyosmium*.
  - `Com01012025_WGS84.shp` [38]: *Shapefile* contenente i confini amministrativi dei comuni italiani, fornito dal sito ufficiale dell'ISTAT e aggiornato a gennaio 2025. Viene utilizzato per benchmark su dati *vettoriali* standard, evitando la complessità del formato *OSM*.

- `GHS_POP_ITALY_100m.tif` [7]: file *raster* in formato *GeoTIFF* contenente la griglia che rappresenta la distribuzione della popolazione italiana con una risoluzione di 100 metri. Questo file è stato generato dall'unione di altri file *raster* scaricati dalla piattaforma GHS-POP di Copernicus, dato che l'Italia viene suddivisa in quattro quadranti differenti.
- **Dataset derivati:** a partire dai dataset provenienti da *OSM*, sono stati estratti e processati i seguenti dataset tematici per le tre città di analisi (Pinerolo, Milano e Roma), specifici per gli scenari di benchmarking. Tutti i file sono stati generati tramite il file Jupyter Notebook `ingestion_and_filtering_data.ipynb` e salvati in formato *GeoParquet*, data la sua efficienza in lettura e scrittura. Per evitare inutili ripetizioni, l'elenco a seguire descrive le tipologie di dataset create. Ciascuna di queste tipologie è stata prodotta per ognuna delle tre aree geografiche di analisi, pertanto, il prefisso `[città]` nel nome del file va inteso come sostituibile con `pinerolo`, `milan` o `rome`.
  - `[città]_buildings.geoparquet`: contiene le geometrie (poligoni) che rappresentano gli edifici. Utilizzato per calcoli di area e analisi spaziali su larga scala.
  - `[città]_bus_stops.geoparquet`: contiene le geometrie (punti) delle fermate degli autobus. Utilizzato in operazioni di join di prossimità.
  - `[città]_hospitals.geoparquet`: include punti e poligoni che identificano ospedali e strutture sanitarie. Utilizzato per analisi di prossimità complesse.
  - `[città]_neighborhoods.geoparquet`: contiene i poligoni delle suddivisioni amministrative o dei quartieri delle città. Utilizzato per aggregazioni spaziali *Point-In-Polygon* [104].
  - `[città]_parks.geoparquet`: include i poligoni che rappresentano parchi pubblici e aree verdi. Utilizzato in operazioni geometriche complesse come l'unione e la differenza.
  - `[città]_restaurants.geoparquet`: contiene le geometrie (punti) di ristoranti e altri locali di ristorazione. È il dataset principale per le query di tipo *Point-In-Polygon*.
  - `[città]_residential_streets.geoparquet`: contiene le geometrie (linee) delle strade classificate come residenziali. Utilizzato in analisi di prossimità multi-layer.
  - `[città]_trees.geoparquet`: include i punti che rappresentano la posizione degli alberi. Utilizzato per analisi di prossimità dettagliate.



## 3. DuckDB, il database in-process con estensione spaziale

Come già descritto nell'introduzione di questa tesi, si analizza il database *DuckDB* come alternativa moderna ai sistemi più tradizionali, in particolare per scenari di analisi interattiva e basata su file.

Una volta spiegato **che cos'è DuckDB**, questo capitolo tratterà le sue **caratteristiche principali**, l'**estensione spaziale** utilizzata durante l'analisi e il suo **ruolo nel progetto**.

### 3.1. Che cos'è DuckDB?

*DuckDB* è un sistema di gestione di database relazionali (RDBMS) SQL di tipo *OLAP* (Online Analytical Processing) [102], open source e *in-process* [40]. A differenza dei database tradizionali come *PostgreSQL* o *MySQL*, che operano come sistemi client-server separati, *DuckDB* funziona come una libreria direttamente all'interno del software "ospitante" (in questo progetto: script Python), permettendo comunque di creare, modificare e applicare query su tabelle [1].

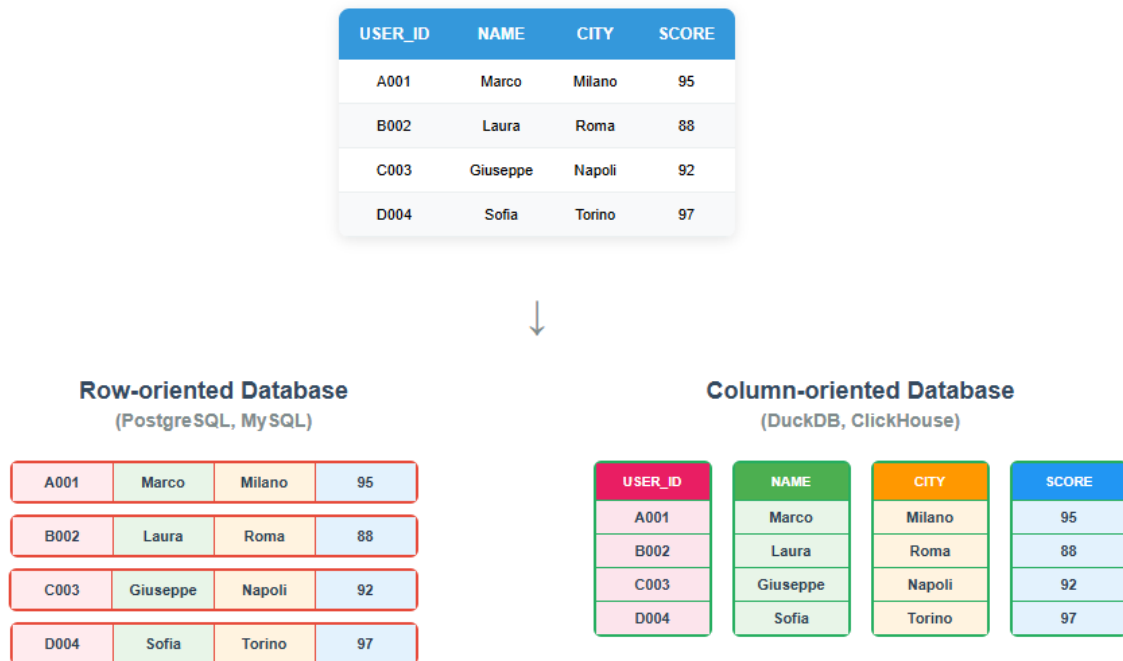
Questo modello è molto simile a quello di *SQLite*, però ha come vantaggio quello di eliminare la necessità di installare, configurare e mantenere un server di database esterno, il che semplifica drasticamente l'architettura per l'analisi dei dati. Inoltre, mentre *SQLite* è ottimizzato per carichi di lavoro transazionali (*OLTP*) [72], *DuckDB* è stato progettato specificatamente per l'analisi *OLAP*, eccellendo così in query complesse che aggregano e analizzano grandi volumi di dati [71].

### 3.2. Caratteristiche principali

L'architettura di *DuckDB* è focalizzata sulla velocità e sulla semplicità d'uso per l'analisi dei dati, soprattutto grazie alle sue caratteristiche distintive:

- **Archiviazione colonnare:** a differenza dei database tradizionali che registrano i dati in righe (*row-oriented*), *DuckDB* memorizza i dati per colonne (*column-oriented*). Questo approccio risulta estremamente proficuo per le query analitiche, visto che richiedono dati appartenenti a più colonne di una determinata tabella, semplificando l'accesso ai dati e riducendo i tempi di risoluzione. Inoltre, a parità di spazio su disco, i database *column-oriented* riescono a salvare più informazioni rispetto a quelli *row-oriented*, grazie ad un efficiente meccanismo di compressione dei dati [67, 71].

## Column-oriented vs Row-oriented Database Storage



**Figura 3.2.1:** Immagine che mette a confronto l’archiviazione di dati *row-oriented* con quella *column-oriented*.

- **Motore d’esecuzione vettorizzato:** è la caratteristica principale alla base delle prestazioni di *DuckDB*. Invece di processare i dati una riga alla volta, il suo motore opera su “vettori”, ossia blocchi di valori provenienti da una colonna. Così facendo, le query vettorizzate riescono a massimizzare l’efficienza della cache della CPU (Central Processing Unit) e a ridurre il carico delle chiamate di funzione, portando a un’esecuzione delle query molto più rapida [44, 72].
- **Nessuna dipendenza esterna:** *DuckDB* è un sistema auto-contenuto, non ha alcuna dipendenza esterna, né per la compilazione né per l’esecuzione. Ciò ne facilita notevolmente la distribuzione, l’installazione (es. nel progetto: `pip install duckdb`) e l’integrazione in qualsiasi compilatore [44].
- **Sistema di estensioni:** molte funzionalità di base possono essere migliorate grazie al meccanismo di estensione flessibile offerto dalla speciale architettura di *DuckDB*. Le estensioni consentono di definire nuovi formati di dati, tipi, sintassi SQL e funzionalità specifiche, come nel caso dell’estensione *Spatial*. È presente una vasta gamma di estensioni grazie ai numerosi contributi da parte degli utenti nella community di *DuckDB* [44, 71].

Oltre a queste sue caratteristiche, ciò che rende *DuckDB* particolarmente potente e performante è la sua integrazione con gli strumenti e le strutture dati moderne:

- **Integrazione zero-copy:** grazie al suo supporto nativo per il protocollo *Apache Arrow* [76], lo scambio di dati tra *DuckDB* e le librerie Python può avvenire in modalità **zero-copy**. Questo significa che quando viene eseguita una query

su dataset in formato *Arrow*, i risultati vengono restituiti nello stesso formato, evitando che i dati vengano duplicati in memoria. Questo meccanismo elimina un enorme sovraccarico di dati, rendendo le analisi ancora più rapide e prestazionali [66].

- **Query su dati remoti:** attraverso estensioni come **httpfs**, *DuckDB* può interrogare file in formato *Parquet* (quindi anche *GeoParquet*) che non si trovano nella memoria del computer locale, ma su un server HTTP o su servizi di cloud storage come Amazon S3 [3]. Questa capacità permette di analizzare dataset di grandi dimensioni direttamente dove sono archiviati, eliminando quasi completamente la necessità di effettuare complesse e lente procedure *ETL* (*Extract, Transform and Load*) per importare i dati da analizzare [42]. L'estensione *httpfs* permette anche la lettura parziale dei dati presenti all'interno di file *Parquet*, scaricando solamente le porzioni del file richieste per eseguire determinate query [43].

### 3.3. L'estensione Spatial

L'importanza e la prevalenza dei dati geospaziali nei contesti dell'analisi di dati odierna hanno portato gli sviluppatori di *DuckDB* a creare e implementare uno strumento adatto per gestire questo tipo di dati. La soluzione è arrivata con lo sviluppo dell'estensione *Spatial*. Questa estensione permette di integrare, trasformare e gestire i dati geospaziali con quelli regolari, grazie all'ecosistema di dati ed estensioni già esistenti per *DuckDB*, portando anche all'introduzione di nuove funzioni e nuovi metodi [45].

Le principali funzionalità dell'estensione *Spatial*, di grande aiuto in questo progetto, sono le seguenti:

- **Supporto allo standard Simple Features:** fornisce un'ampia gamma di funzioni SQL spaziali che seguono il modello standard *Simple Features for SQL* dell'*Open Geospatial Consortium (OGC)* [107], il quale permette di accedere, utilizzare e costruire elementi di tipo **GEOMETRY** provenienti dalle classiche geometrie vettoriali, ossia *punti*, *polilinee* e *poligoni* [87]. Le funzioni derivanti dal modello *Simple Features* più utilizzate nel progetto sono **ST\_Within**, **ST\_DWithin**, **ST\_Area**, **ST\_Transform** e **ST\_Union\_Agg**.
- **Integrazione di librerie geospaziali:** tra le altre molteplici funzioni aggiuntive che l'estensione *Spatial* fornisce, la maggior parte si basa su tre librerie geospaziali fondamentali, ossia **GEOS** [15], **GDAL** [12] e **PROJ** [25]. Queste librerie non forniscono direttamente funzioni SQL, bensì algoritmi computazionali, funzionalità di importazione ed esportazione in diversi formati, e conversioni di coordinate tra differenti *CRS* [45].
- **Lettura e scrittura diretta di file geospaziali:** una delle caratteristiche più utili e funzionali dell'estensione è la capacità di poter leggere e scrivere dati direttamente in molteplici formati di file geospaziali tramite l'utilizzo della funzione **ST\_Read**, introdotta con l'integrazione della libreria *GDAL* [41]. Nello specifico di questo progetto, la capacità di poter interrogare file *GeoParquet* e *Shapefile* direttamente su script è stata fondamentale per alleggerire il flusso lavorativo delle analisi su *DuckDB*.

## 3.4. Ruolo nel progetto

All'interno di questo studio, *DuckDB* ha rappresentato la figura dello stack tecnologico moderno e *file-based*. È stato impiegato per svolgere le seguenti attività:

- Lavorare come “motore” sottostante della libreria *quackosm* per effettuare l'ingestione e il filtraggio di dati dai file *PBF* di *OpenStreetMap* in tempi estremamente rapidi.
- Operare nativamente per svolgere le medesime attività sui file *Shapefile* per i dati *vettoriali puri*.
- Eseguire correttamente tutte e sei le query dei casi d'uso 3 e 4 direttamente sui file *GeoParquet*, dimostrando l'esistenza di un approccio di analisi *database-less*.

Per il caso d'uso 5, *DuckDB* è stato escluso dall'analisi *vettoriale-raster* a causa del suo mancato supporto nativo e della sua conseguente incompatibilità con i dati *raster*.

L'analisi condotta su *DuckDB* ha inoltre portato a una delle scoperte principali della tesi: le elevate prestazioni derivano strettamente da una configurazione precisa e da un'accurata conoscenza di ogni specifica dei metadati in uso, in particolare il *CRS* per i dati geospaziali, evidenziando un *trade-off* tra la velocità degli strumenti moderni e la maturità del loro ecosistema.

## 4. PostGIS, l'estensione spaziale di PostgreSQL

All'interno di questo studio comparativo, *PostGIS* rappresenta la tecnologia riconosciuta a livello mondiale come lo standard per la gestione di dati geospaziali in un ambiente di database relazionali, un ruolo che è stato confermato anche in recenti studi su architetture per "Big Data" [109].

In questa parte di tesi, viene descritto **che cos'è PostGIS** e analizzata la sua **architettura** con le sue **caratteristiche principali**, spiegando anche quale sia il **ruolo nel progetto** di questa tecnologia.

### 4.1. Che cos'è PostGIS?

*PostGIS* non è un database a sé stante, bensì un'estensione per il database relazionale ad oggetti *PostgreSQL*. La sua funzione è quella di "potenziare" il database nativo, aggiungendo il supporto per tipi di dati, funzioni, query e indici spaziali. In questo modo, trasforma un database general-purpose [69] in un *Geographic Information System (GIS)* [28] funzionante e conforme agli standard geospaziali.

*PostGIS* viene continuamente aggiornato da oltre due decenni ed è molto apprezzato per la sua stabilità, la vasta gamma di funzionalità che integra e l'implementazione di diversi standard geospaziali, come *Geography Markup Language (GML)* e *Well-Known Text (WKT)* [16].

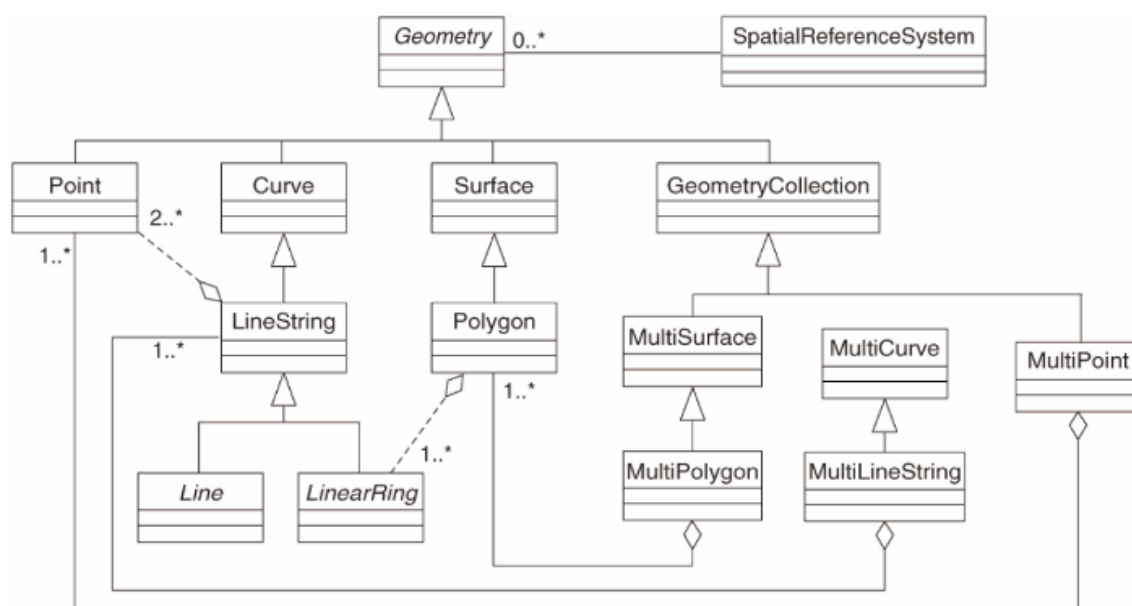
### 4.2. Architettura e caratteristiche principali

A differenza di un sistema *in-process* come *DuckDB*, *PostGIS* opera all'interno dell'architettura client-server di *PostgreSQL*. Questo modello è progettato per ambienti multi-utente e centralizzati, dove la consistenza, la sicurezza e la gestione concorrente dei dati sono fattori prioritari.

Le seguenti caratteristiche distintive di *PostGIS* sono il motivo fondamentale per cui il suo utilizzo è ancora oggi fortemente consigliato:

- **Vasta libreria di funzioni spaziali:** *PostGIS* offre centinaia di funzioni per l'analisi, la costruzione e l'elaborazione di geometrie spaziali. Queste includono operazioni di misurazione (aree, distanze, lunghezze), relazioni spaziali (intersezioni, prossimità), geoprocessing (buffer, unioni, differenze) e molte altre tipologie [57].

- **Supporto a dati raster:** nonostante sia stato sviluppato per i dati *vettoriali*, *PostGIS* presenta un potente supporto anche per i dati *raster* [60], permettendo di salvare e analizzare griglie di dati direttamente all'interno del database con l'uso dell'eseguibile *raster2pgsql* [55]. Grazie a questa integrazione, si possono effettuare le analisi comparative tra le due diverse tipologie di dati.
- **Indicizzazione spaziale avanzata:** la caratteristica tecnica che garantisce prestazioni migliori di *PostGIS* nelle query spaziali è l'uso di indici spaziali, dato che permettono di utilizzare dataset di grandi dimensioni senza problematiche. Questi indici spaziali si basano sulla struttura dati **Generalized Search Tree (GiST)** [52], che organizza i dati in una struttura ad albero completamente bilanciata. Quando viene eseguita una query, il database non confronta la geometria di ricerca con ogni singolo record, ma usa l'indice corrispondente per escludere rapidamente tutte le sezioni di dati non pertinenti, limitando la ricerca a un piccolo sottoinsieme e riducendo i tempi d'esecuzione [54].
- **Conformità agli standard OGC:** come *DuckDB*, *PostGIS* implementa le specifiche *Simple Features for SQL* dell'*Open Geospatial Consortium (OGC)*, integrando due diversi tipi di dati, **geometry** e **geography**, derivanti dal modello **GEOMETRY**. Questo garantisce un ottimo grado di interoperabilità con altre tecnologie *GIS* e una corretta standardizzazione delle funzioni, facendo sì che i valori geometrici rappresentino situazioni realistiche [53].



**Figura 4.2.1:** In questa figura viene mostrato il modello *Simple Features* per tutti i dati di tipo *geometry* [87].

- **Identificatore spaziale:** le geometrie adottate non hanno come sistema di riferimento un classico *CRS*, ma un identificatore spaziale unico chiamato **Spatial Reference Identifier (SRID)**, a cui sono associati un sistema di coordinate, una tolleranza e una risoluzione specifici [31].
- **Integrazione della topologia:** una delle funzionalità più avanzate di *PostGIS* è la sua implementazione delle regole topologiche. A differenza delle geometrie

vettoriali semplici, dove i *poligoni* adiacenti memorizzano i loro confini comuni, causando la duplicazione dei dati, una struttura topologica registra il confine condiviso una sola volta come un **edge** (bordo) [56] condiviso, garantendo così l'integrità di entrambi gli oggetti. Oltre a questo vantaggio, l'implementazione fornisce un elevato numero di funzioni per la creazione, modifica, importazione ed esportazione di topologie [62].

### 4.3. Ruolo nel progetto

In questa tesi, *PostGIS* è stato utilizzato come stack tecnologico di riferimento per valutare la correttezza numerica e la robustezza delle due alternative più moderne. Il suo flusso di lavoro ha richiesto una fase preliminare di ingestione dei dati, in cui i file `.osm.pbf`, `.shp` e `.tif` sono stati caricati e strutturati in tabelle del database tramite i corrispondenti strumenti dedicati *osm2pgsql*, *shp2pgsql* e *raster2pgsql*. Tutte le operazioni di benchmark sono state eseguite tramite query SQL specifiche su un database pre-popolato, simulando correttamente un tipico ambiente di produzione in cui i dati sono gestiti centralmente. L'analisi dei risultati ha confermato la sua affidabilità nei calcoli geometrici complessi, ma ha anche evidenziato un chiaro *trade-off* prestazionale, mostrando significative lacune in termini di velocità nelle operazioni di join di prossimità rispetto agli approcci *file-based* e *in-memory*.



## 5. GeoPandas, la libreria Python per dati geospaziali

La terza e ultima tecnologia analizzata per lo studio di questa tesi è *GeoPandas*, che rappresenta l'approccio all'analisi geospaziale basato interamente sul linguaggio Python e sull'elaborazione dei dati in memoria (*in-memory*).

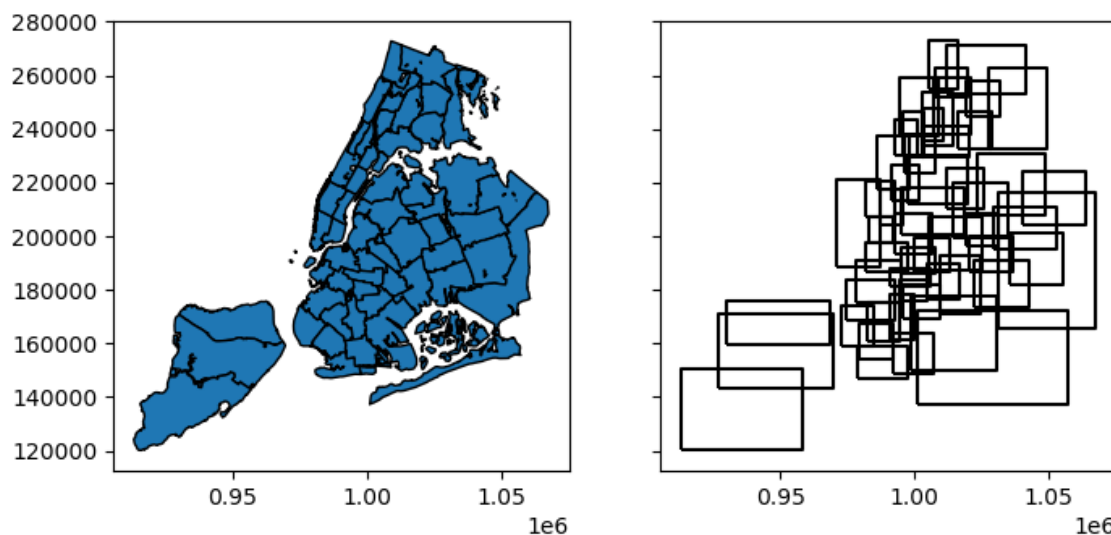
Questo capitolo introduce **che cos'è GeoPandas**, in modo tale da descrivere il suo **ecosistema** e spiegare il suo **ruolo nel progetto**.

### 5.1. Che cos'è GeoPandas?

*GeoPandas* è una libreria open source che estende le capacità di un'altra libreria Python per l'analisi di dati chiamata *pandas*, permettendo di lavorare con i dati geospaziali e le loro geometrie. Il suo obiettivo è rendere l'analisi di dati geospaziali in Python semplice e intuitiva, combinando le capacità delle strutture dati di *pandas* con le funzionalità di manipolazione spaziale [11].

La struttura dati alla base di *GeoPandas* è il **GeoDataFrame**, una sottoclasse del **DataFrame** di *pandas* da cui ha ereditato tutte le funzionalità, ma con l'aggiunta di una o più colonne **geometry**. Questa colonna, di tipo **GeoSeries**, può contenere dati geometrici (*punti, linee, poligoni*) e permette di eseguire operazioni spaziali direttamente sulla tabella di dati [34].

Un aspetto fondamentale per le prestazioni, spesso non notato dall'utente, è l'uso di un indice spaziale. Quando si eseguono operazioni che mettono in relazione due layer, come un join spaziale, confrontare ogni geometria con tutte le altre sarebbe computazionalmente insostenibile. Per risolvere la questione, *GeoPandas* utilizza internamente un indice spaziale basato su una struttura dati ad albero, chiamata **R-tree** [105]. Questo indice organizza le geometrie in base alle loro scatole di delimitazione (**bounding box**) [98], permettendo di escludere a priori e con un costo minimo un numero enorme di confronti non necessari. Molte delle funzioni più performanti di *GeoPandas*, come **sjoin**, **clip** ed **overlay**, sfruttano automaticamente questo meccanismo per accelerare drasticamente le query [37].



**Figura 5.1.1:** Immagine che mostra le geometrie originali (figura a sinistra) e le loro *bounding box* (figura a destra) [37].

## 5.2. L'ecosistema GeoPandas

La caratteristica più importante di *GeoPandas* risiede nell'agglomerare tutte le capacità di diverse librerie Python, costantemente aggiornate e migliorate, all'interno di un ecosistema scientifico modulare [32]. Ognuna di queste librerie ha un compito preciso:

- *pandas* [79]: fornisce la struttura dati *DataFrame* e tutte le funzionalità utili per la manipolazione di dati tabulari e multidimensionali, come il filtraggio, l'aggregazione, il join di oggetti e molte altre.
- *Shapely* [51]: permette di manipolare e analizzare gli oggetti geometrici nel piano Cartesiano. *Shapely* è un package Python per la libreria C/C++ *GEOS* (*Geometry Engine - Open Source*) [15], che è lo stesso motore geometrico utilizzato da *PostGIS*. Grazie a questa dipendenza, i calcoli geometrici eseguiti da *GeoPandas* risultano numericamente coerenti con quelli dell'estensione spaziale di *PostgreSQL*.
- *pyogrio* [93]: questa libreria, introdotta nel 2020, permette di leggere e di scrivere ogni tipologia di dato geometrico nei *GeoDataFrame* di *GeoPandas*. Inoltre, visto che la colonna *geometry* è opzionale nei *GeoDataFrame*, è anche possibile leggere e scrivere dati non spaziali. *pyogrio* è più veloce sia a leggere che a scrivere dati nei file rispetto a *Fiona*, un'altra libreria Python che svolge la stessa funzione, grazie al collegamento pre-compilato per *GDAL/OGR*, che le permette la lettura e la scrittura di record di dati a blocchi.
- *Fiona* [48]: gestisce la lettura e la scrittura riga per riga di file contenenti dati *vettoriali*, scambiando memoria e velocità per semplicità e affidabilità. È una libreria che fa da interfaccia per la libreria *GDAL/OGR*, così da garantire le sue funzionalità anche per i dati geospaziali. Questo permette a *GeoPandas* di leggere e scrivere in diversi formati, come *GeoJSON*, *Shapefile* e *GeoPackage* [50].

- *pyproj* [94]: è l'interfaccia Python per la libreria *PROJ* (*cartographic projections and coordinate transformations library*) [25], con cui si gestiscono i *CRS* e le proiezioni cartografiche degli oggetti di tipo *GeoSeries*. Ogni operazione di trasformazione (o riproiezione) delle coordinate in *GeoPandas* viene eseguita tramite *pyproj*.

Bisogna anche sottolineare che inizialmente le operazioni geometriche in *GeoPandas*, sebbene fossero corrette, potevano risultare lente su grandi dataset. Per risolvere questa problematica, fu creata *PyGEOS* [92], una libreria esterna usata per reimplementare le funzioni geometriche in modo vettorizzato. Ciò permetteva di processare interi array di geometrie in un solo passaggio, ottenendo un enorme guadagno prestazionale. A partire dalla versione 2.0 di *Shapely*, il codice di *PyGEOS* è stato interamente integrato nel package Python aggiornato, rendendo di conseguenza le ultime versioni di *GeoPandas* molto più veloci di quelle passate ed evitando ulteriori installazioni di librerie all'utente [35].

Questo cambiamento è stato sicuramente un fattore chiave per le eccellenti prestazioni di *GeoPandas* in molti dei benchmark di questa tesi.

## 5.3. Ruolo nel progetto

Nello studio, *GeoPandas* ha raffigurato lo stack tecnologico di analisi *in-memory* e basato interamente su script Python. Il suo processo lavorativo non prevede alcuna fase di ingestione di dati in un database, perché i dati vengono letti direttamente dai file (in questo caso specifico, file *.geoparquet*), caricati all'interno di un *GeoDataFrame* e processati interamente nella RAM del sistema in uso.

Questo approccio si è dimostrato performante in specifici casi d'uso, in particolare nei join di prossimità, dove la funzione ottimizzata *sjoin\_nearest* ha portato *GeoPandas* a risolvere le operazioni quasi mille volte più velocemente rispetto alle altre due tecnologie. Allo stesso tempo, l'analisi ha evidenziato come l'elaborazione interamente *in-memory* possa comportare un *bottleneck* in operazioni computazionalmente intensive su dataset di grandi dimensioni, dove gli strumenti con database ottimizzati come *PostGIS* mantengono un vantaggio significativo.



## 6. I casi d'uso, cinque scenari di benchmark

L'analisi comparativa tra *DuckDB*, *PostGIS* e *GeoPandas* si è svolta tramite l'identificazione di cinque diversi *casi d'uso* [99], utili ad affrontare molteplici scenari ed operazioni, integrando anche librerie Python di supporto.

Per ogni *caso d'uso* sono stati indicati il loro obiettivo, le tecnologie messe a confronto, un esempio o le operazioni specifiche dello scenario, e le metriche campionate. Inoltre, sono stati divisi in base al tipo di dato utilizzato per l'analisi:

- Il primo e il secondo sono gli unici adatti a tutti e tre i tipi di dati.
- Il terzo e il quarto sono stati definiti per i dati di *OSM*.
- Il quinto è stato ideato esclusivamente per combinare dati *vettoriali puri* con dati *raster*.

Di conseguenza, le sezioni di questo capitolo si suddividono in **casi d'uso per dati vettoriali OSM** e **casi d'uso per dati vettoriali puri e raster**.

### 6.1. Casi d'uso per dati vettoriali OSM

Per confrontare le tre tecnologie in analisi su dati *vettoriali OpenStreetMap*, sono stati definiti quattro *casi d'uso* principali.

#### 6.1.1. Caso d'uso 1: Ingestione Dati

- **Obiettivo:** misurare l'efficienza in termini di costi (tempo e spazio) nel caricare un intero file PBF in una struttura dati o in un sistema che lo renda interrogabile da una tecnologia, utilizzando una libreria specifica.
- **Tecnologie a confronto:**
  - DuckDB + quackosm
  - PostGIS + osm2pgsql
  - GeoPandas + pyosmium
- **Esempio:** importare il file `italy-latest.osm.obf` in un database PostGIS tramite lo strumento `osm2pgsql`.
- **Metriche:** tempistiche totali di elaborazione, throughput, dimensione e coerenza dei dati finalizzati.

### 6.1.2. Caso d'uso 2: Filtraggio ed Estrazione Dati

- **Obiettivo:** misurare la velocità delle librerie nell'estrarre un sottoinsieme specifico di dati da una fonte già ingerita o direttamente da un file completamente grezzo, in modo tale da renderli disponibili alla Tecnologia in analisi.
- **Tecnologie a confronto:**
  - DuckDB + quackosm
  - PostGIS + osm2pgsql
  - GeoPandas + pyosmium
- **Esempio:** estrarre tutti gli edifici (tag: `building=yes`) per una specifica area geografica e salvarli in un formato analitico, come GeoParquet.
- **Metriche:** tempistiche totali di elaborazione, throughput, facilità di implementazione, dimensione e coerenza dei dati finalizzati.

### 6.1.3. Caso d'uso 3: Analisi su Singola Tabella

- **Obiettivo:** valutare le performance delle tecnologie nell'eseguire calcoli geometrici e attributivi, che non richiedono l'uso di join (es.: calcolo dell'area media), su dati già processati e puliti.
- **Tecnologie a confronto:**
  - DuckDB con estensione Spatial
  - PostGIS
  - GeoPandas
- **Operazioni di caso d'uso specifiche:**

Per valutare le performance in questo scenario, ho deciso di eseguire le seguenti operazioni su dataset di dimensioni diverse (Pinerolo, Milano e Roma):

  - **Top 10 Aree più Grandi:** questa operazione misura le performance del sistema nell'eseguire un calcolo geometrico (`ST_Area`) su ogni feature, seguito da un ordinamento crescente (`ORDER BY`) e da una selezione dei primi N risultati (`LIMIT`). Simula un caso d'uso comune di classificazione e identificazione di outlier dimensionali in un dataset. Testa l'efficienza del calcolo geometrico e dell'algoritmo di ordinamento.
  - **Creazione di un Buffer e Calcolo Area Conseguente:** questa operazione valuta la capacità di eseguire operazioni geometriche a catena. Per ogni edificio viene creato un buffer di 10 metri (`ST_Buffer`) e successivamente viene calcolata l'area coperta da tutti i buffer (`ST_Area + SUM`). Serve a rappresentare analisi di impatto, prossimità e valutazione delle zone di influenza.
  - **Insieme di Ristoranti che NON si trovano nelle Vicinanze di Fermate di Bus:** questa operazione testa la capacità del sistema nell'eseguire

un anti-join spaziale per identificare tutte le feature di un dataset (ristoranti) che non hanno alcuna feature di un secondo dataset (fermate di bus) entro una specifica distanza di prossimità. L'implementazione si basa su una sottoquery con `NOT EXISTS` combinata con la funzione `ST_DWithin`. Simula scenari di analisi di accessibilità o di identificazione di "gap" nei servizi, valutando l'efficienza del motore di join per query di prossimità negative tra due tabelle differenti.

- **Metriche:** tempistiche di esecuzione della query, utilizzo della memoria (se possibile misurarlo) e coerenza dei risultati.

#### 6.1.4. Caso d'uso 4: Join Spaziale Complesso

- **Obiettivo:** testare le performance delle Tecnologie su operazioni computazionalmente intensive che richiedono l'interazione tra due dataset diversi (es.: calcolare la prossimità tra punti e poligoni).

- **Tecnologie a confronto:**

- DuckDB con estensione Spatial
- PostGIS
- GeoPandas

- **Operazioni di caso d'uso specifiche:**

Per valutare le performance in questo scenario, ho deciso di eseguire le seguenti operazioni di join spaziale, utilizzando dataset di città di dimensioni diverse (Pinerolo, Milano e Roma):

- **Ristoranti per Quartiere:** operazione di Point-In-Polygon. Misura l'efficienza del join spaziale nel contare quanti punti (ristoranti con tag: `amenity=restaurant`) ricadono all'interno di ciascun poligono (quartieri con tag: `boundary=administrative, admin_level=9`). Viene usata per aggregare dati puntuali in base all'area.
- **Conteggio Alberi e Somma Lunghezza Strade Residenziali Vicine a Ospedali:** questa operazione simula una query analitica complessa e multi livello. Richiede una serie di passaggi:
  - \* Selezionare le strade residenziali, dove viene isolato un sottoinsieme di strade in base al tag corretto (tag: `highway=residential`).
  - \* Eseguire il primo join di prossimità, per identificare le strade residenziali che si trovano entro 100 metri dagli ospedali.
  - \* Svolgere il secondo join di prossimità, che viene applicato al sottoinsieme specifico di queste strade per contare tutti gli alberi (tag: `natural=tree`) nelle vicinanze.
  - \* Aggregare tutte le lunghezze delle strade del sottoinsieme, per ottenerne la lunghezza totale tramite sommatoria.

Testa la capacità del sistema di gestire sottoquery, join spaziali a catena e aggregazioni multiple (`COUNT + SUM`) su dati filtrati, analizzando uno scenario di analisi territoriale avanzato.

- **Calcolo Area NON Coperta da Parchi:** questa operazione valuta le performance di sistema su operazioni geometriche costruttive e distruttive su larga scala. Richiede l'unione di tutti i poligoni dei parchi (tag: `leisure=park`) in un'unica geometria complessa (`ST_Union`) e la successiva sottrazione (`ST_Difference`) di questa geometria dall'area del quartiere. Questa operazione dovrebbe risultare computazionalmente pesante.
- **Metriche:** tempistiche di esecuzione, correttezza dei risultati e complessità totale del codice.

## 6.2. Casi d'uso per dati vettoriali puri e raster

Per comparare le tre architetture anche su dati *vettoriali puri* e *raster*, è stato definito un caso d'uso principale ed estesi due casi d'uso già introdotti ed utilizzati per dati *OpenStreetMap*.

Come già anticipato, per tutti e tre i *casi d'uso*, *DuckDB* non viene usata come tecnologia di benchmark per i dati *raster*, vista la sua mancanza di compatibilità nella gestione di questi ultimi.

### 6.2.1. Caso d'uso 1: Ingestione Dati

- **Obiettivo:** misurare l'efficienza in termini di costi (tempo e spazio) nel leggere un file vettoriale o raster dal disco e renderlo disponibile per l'analisi, sia in memoria che in un database.
- **Tecnologie a confronto:**
  - Per dati vettoriali:
    - \* DuckDB
    - \* PostGIS + shp2pgsql
    - \* GeoPandas
  - Per dati raster:
    - \* PostGIS + raster2pgsql
    - \* Python con libreria rioxarray
- **Esempio:** caricare lo Shapefile dei comuni ISTAT o il file GeoTIFF della popolazione GHS-POP di Copernicus.
- **Metriche:** tempistiche totali di elaborazione, throughput e facilità di implementazione.

### 6.2.2. Caso d'uso 2: Filtraggio ed Estrazione Dati

- **Obiettivo:** misurare l'efficienza nell'estrarre un sottoinsieme di dati basato su un filtro spaziale (clipping) o attributivo.
- **Tecnologie a confronto:**
  - Per dati vettoriali:

- \* DuckDB
  - \* PostGIS
  - \* GeoPandas
- Per dati raster:
  - \* PostGIS
  - \* Python con libreria rioxarray
- **Esempio:** estrarre tutti i comuni di una regione dal dataset nazionale in formato Shapefile o ritagliare il file raster nazionale della popolazione, usando il confine di una città.
- **Metriche:** tempistiche totali di elaborazione, dimensione dei dati finalizzati, facilità di implementazione e coerenza dei risultati.

### 6.2.3. Caso d'uso 5: Analisi Vettoriale-Raster

- **Obiettivo:** valutare le capacità e le performance delle diverse tecnologie nell'effettuare l'estrazione di statistiche da dati raster basate su zone definite da dati vettoriali.
- **Tecnologie a confronto:**
  - PostGIS con estensione postgis\_raster
  - Python con librerie rioxarray e GeoPandas
- **Operazione di caso d'uso specifica:**
  - **Popolazione Totale per Comune:** utilizzando i confini amministrativi dei comuni ISTAT (dati vettoriali) e il raster della popolazione GHS-POP (dati raster), calcolare la popolazione totale per ogni comune di una data regione. L'operazione richiede di sommare i valori di tutti i pixel del raster che ricadono all'interno di ogni poligono del comune.
- **Metriche:** tempistiche di esecuzione, correttezza dei risultati e complessità totale dell'implementazione.



## 7. I risultati ottenuti

In questo capitolo della tesi vengono presentati e discussi i risultati emersi dal framework di benchmarking, che sono stati ottenuti misurando le prestazioni quantitative, ossia i tempi d'esecuzione, e valutando gli aspetti qualitativi, come la coerenza numerica e la robustezza di ogni stack tecnologico di fronte a *casi d'uso* realistici e a dati complessi.

I dati raccolti non definiscono una gerarchia tra le architetture, ma rivelano una serie di *trade-off*, che dimostrano come la scelta dello strumento ottimale dipenda fortemente dal compito da svolgere.

La trattazione di tutti gli esiti è suddivisa in tre sezioni principali:

- I **risultati nello specifico**, dove vengono mostrati e commentati direttamente i dati ottenuti dall'esecuzione di ogni *caso d'uso*.
- I **consigli applicativi**, che forniscono una guida pratica per la scelta della tecnologia più adatta in base a specifici contesti lavorativi.
- Le **scoperte significative**, in cui sono elencati e descritti i risultati più importanti emersi dall'analisi comparativa, permettendo di contestualizzare i compromessi tra velocità, maturità e usabilità degli strumenti.

### 7.1. I risultati nello specifico

Qui di seguito sono presentate e commentate le tabelle riassuntive di tutti i risultati ottenuti con lo svolgimento di ogni *caso d'uso*, suddivise per area geografica di analisi in modo tale da valutare la scalabilità delle tecnologie con dati crescenti in complessità e dimensione. Nello specifico, le tabelle riportano le operazioni eseguite, le tecnologie in analisi, i tempi medi di esecuzione in condizioni di “**hot cache**”, che rappresentano le prestazioni a regime del sistema [10], e i risultati numerici finali. Inoltre, per determinate operazioni, sono presenti dati riguardanti metriche aggiuntive, come il **throughput** [97] per le operazioni di I/O e, dove applicabile, l'utilizzo medio della CPU e il consumo al picco della RAM.

7.1.1. Casi d’uso 1 e 2: Ingestione e Filtraggio Dati vettoriali OSM

Le prime fasi di ogni flusso di analisi geospaziale riguardano l’acquisizione e il filtraggio dei dati. La tabella 7.1.1.1 confronta i diversi approcci dei tre stack tecnologici per effettuare l’estrazione degli edifici di Milano da fonti *OSM*.

| Operazione                     | Tecnologia           | Tempo (s)   | Risultato     | Throughput (MB/s) |
|--------------------------------|----------------------|-------------|---------------|-------------------|
| Estrazione da PBF (1.9 GB)     | DuckDB + quackosm    | 0.22        | 62133 edifici | 8636              |
| Query da Database (19 GB)      | PostGIS + osm2pgsql  | 0.40 + 1840 | 62127 edifici | 10.33             |
| Parsing PBF in-memory (256 MB) | GeoPandas + pyosmium | 360.10      | 62075 edifici | 0.71              |

**Tabella 7.1.1.1:** Confronto prestazioni per Ingestione e Filtraggio edifici di Milano (dati vettoriali OSM)

**Commento alla tabella 7.1.1.1:** si può notare come l’approccio *DuckDB* con la libreria *quackosm* sia il più efficiente per l’estrazione diretta di dati da file *PBF*, dato che risulta quasi **1650 volte più veloce** dello stack *GeoPandas* con *pyosmium*, beneficiando della cache di sistema. Questo dimostra come una libreria a basso livello ottimizzata sia superiore al parsing interamente in Python per l’I/O di file complessi.

*PostGIS* mostra ottimi tempi di interrogazione su un database già popolato, ma non per quanto riguarda l’ingestione iniziale, che ha richiesto circa **31 minuti (1840 secondi)** per essere completata e ha incrementato le dimensioni del disco di **19 GB**, risultando una fase parecchio costosa.

Non sono state diversificate in tabella le fasi di ingestione e filtraggio, perché sia *DuckDB* che *GeoPandas* usufruiscono di due librerie che permettono di fare un’estrazione diretta dei dati *vettoriali OSM* senza dover effettuare prima la loro ingestione.

### 7.1.2. Casi d'uso 1 e 2: Ingestione e Filtraggio Dati vettoriali puri

Le stesse fasi sono state replicate su un file in formato *vettoriale*, lo *Shapefile* dei comuni italiani, in modo tale da isolare le prestazioni dal formato *PBF*. Nella tabella 7.1.2.1 vengono confrontate le tre tecnologie sia per l'ingestione che per il filtraggio dei dati.

| Operazione          | Tecnologia          | Tempo (s)     | Risultato   | Throughput (MB/s) |
|---------------------|---------------------|---------------|-------------|-------------------|
| Ingestione (105 MB) | DuckDB Spatial      | <b>0.33</b>   | 7896 comuni | <b>318.18</b>     |
|                     | PostGIS + shp2pgsql | 21            | 7896 comuni | 5                 |
|                     | GeoPandas           | 0.34          | 7896 comuni | 308.82            |
| Filtraggio          | DuckDB Spatial      | 0.019         | 1180 comuni | N/A               |
|                     | PostGIS             | 0.075         | 1180 comuni | N/A               |
|                     | GeoPandas           | <b>0.0009</b> | 1180 comuni | N/A               |

**Tabella 7.1.2.1:** Confronto prestazioni per Ingestione e Filtraggio comuni italiani (dati vettoriali)

**Commento tabella 7.1.2.1:** per quanto riguarda l'ingestione di dati *vettoriali*, *DuckDB* e *GeoPandas* sono equivalentemente veloci, leggendo i file in circa **0.3 secondi**, segno che l'I/O su file **Shapefile** diventa l'unico vero bottleneck.

Nel filtraggio, *GeoPandas* esegue l'operazione *in-memory* quasi istantaneamente, dimostrandosi la soluzione più performante per effettuare analisi su dati già caricati in memoria, evitando anche costi di accesso a disco.

In entrambe le fasi, *PostGIS* mantiene la sua solidità, ma soffre le necessarie tempistiche di importazione dei dati nel database (**≈21 secondi**).

### 7.1.3. Casi d'uso 1 e 2: Ingestione e Filtraggio Dati raster

Infine, sono state testate le capacità di gestione di dati *raster* in formato *GeoTIFF*, confrontando solo *PostGIS* e lo stack Python, dato che *DuckDB* non è compatibile con essi. La tabella 7.1.3.1 contiene comunque le celle di *DuckDB* per sottolineare questa informazione.

| Operazione           | Tecnologia             | Tempo (s)   | Risultato     | Throughput (MB/s) |
|----------------------|------------------------|-------------|---------------|-------------------|
| Ingestione (2.97 GB) | DuckDB Spatial         | N/A         | N/A           | N/A               |
|                      | PostGIS + raster2pgsql | 370         | N/A           | 8.03              |
|                      | Python (rioxarray)     | <b>0.03</b> | N/A           | <b>99000</b>      |
| Filtraggio           | DuckDB Spatial         | N/A         | N/A           | N/A               |
|                      | PostGIS                | <b>7.40</b> | 1.26M persone | N/A               |
|                      | Python (rioxarray)     | 7.91        | 1.26M persone | N/A               |

**Tabella 7.1.3.1:** Confronto prestazioni per Ingestione e Filtraggio popolazione del comune di Milano (dati raster)

**Commento tabella 7.1.3.1:** l'approccio Python con la libreria *rioxarray* è molto più veloce nella fase di ingestione di file *raster*, in quanto si limita a leggerne solo i metadati, mentre *PostGIS* deve importare e “tassellare” l'intero *raster* nel database, impiegandoci circa **6 minuti** e aumentando anche la dimensione dei dati su disco. Tuttavia, nel filtraggio, le prestazioni sono sorprendentemente simili, dimostrando quanto le due tecnologie siano altamente competitive, grazie all'integrazione nei loro ecosistemi di librerie e strumenti ottimizzati per la gestione di dati *raster*.

7.1.4. Caso d’uso 3: Analisi su Singola Tabella

Questo *caso d’uso* valuta le prestazioni delle tecnologie in analisi eseguendo calcoli geometrici e anti-join sui dataset di tre diverse città di grandezza crescente (Pinerolo, Milano, Roma).

Risultati per Pinerolo

| Operazione                                   | Tecnologia     | Tempo (s)     | Risultato            | CPU (%)<br>+<br>RAM (MB)  |
|----------------------------------------------|----------------|---------------|----------------------|---------------------------|
| 3.1. Top 10 Aree più Grandi                  | DuckDB Spatial | 0.017         | 10 edifici           | <b>16.8</b>   126.87      |
|                                              | PostGIS        | <b>0.017</b>  | 10 edifici           | 20.9   <b>82.26</b>       |
|                                              | GeoPandas      | 0.053         | 10 edifici           | 21.8   127.77             |
| 3.2. Area Totale con Buffer                  | DuckDB Spatial | <b>0.120</b>  | 7.05 km <sup>2</sup> | <b>18.6</b>   129.95      |
|                                              | PostGIS        | 0.144         | 7.05 km <sup>2</sup> | 18.7   <b>82.32</b>       |
|                                              | GeoPandas      | 0.176         | 7.05 km <sup>2</sup> | 19.1   132.31             |
| 3.3. Ristoranti NON vicino a Fermate del Bus | DuckDB Spatial | 0.062         | 15 ristoranti        | 25.8   150.84             |
|                                              | PostGIS        | <b>0.0008</b> | 15 ristoranti        | <b>3.2</b>   <b>82.34</b> |
|                                              | GeoPandas      | 0.005         | 15 ristoranti        | 14.4   122.90             |

Tabella 7.1.4.1: Risultati dettagliati per l’Analisi su Singola Tabella di Pinerolo

**Commento tabella 7.1.4.1:** sui dataset di piccole dimensioni, le prestazioni sono molto competitive. *DuckDB* e *PostGIS* risultano quasi identici nel calcolo delle aree classico (operazione 3.1), mentre *PostGIS* è l’unico estremamente veloce nell’anti-join (operazione 3.3) grazie ai suoi indici ottimizzati. *GeoPandas* rimane comunque un’ottima alternativa per tutte e tre le operazioni.

Le metriche di sistema rivelano che *PostGIS* mantiene un consumo RAM stabile e contenuto (**≈82 MB**) su tutte le operazioni, risultando il più efficiente tra le tre tecnologie, dato che *DuckDB* e *GeoPandas* utilizzano tra **130** e **150 MB**.

Nella terza operazione, *PostGIS* raggiunge un utilizzo della CPU minimo (**3.2%**), confermando che gli indici spaziali riducono drasticamente il carico computazionale rispetto alle tecnologie che usufruiscono di elaborazioni completamente *in-memory*.

Risultati per Milano

| Operazione                                         | Tecnologia     | Tempo (s)    | Risultato             | CPU (%)<br>+<br>RAM (MB)   |
|----------------------------------------------------|----------------|--------------|-----------------------|----------------------------|
| 3.1. Top 10 Aree<br>più Grandi                     | DuckDB Spatial | <b>0.188</b> | 10 edifici            | 19.2   139.34              |
|                                                    | PostGIS        | 0.196        | 10 edifici            | 31.1   <b>82.42</b>        |
|                                                    | GeoPandas      | 0.869        | 10 edifici            | <b>16.1</b>   351.81       |
| 3.2. Area Totale<br>con Buffer                     | DuckDB Spatial | 2.08         | 111.4 km <sup>2</sup> | 17.6   139.93              |
|                                                    | PostGIS        | <b>1.34</b>  | 111.5 km <sup>2</sup> | 25.3   <b>82.43</b>        |
|                                                    | GeoPandas      | 2.93         | 111.5 km <sup>2</sup> | <b>16.7</b>   377.72       |
| 3.3. Ristoranti<br>NON vicino a<br>Fermate del Bus | DuckDB Spatial | 0.077        | 2135<br>ristoranti    | 24.0   164.13              |
|                                                    | PostGIS        | 10.49        | 2135<br>ristoranti    | <b>16.4</b>   <b>83.67</b> |
|                                                    | GeoPandas      | <b>0.048</b> | 2136<br>ristoranti    | 30.1   369.76              |

**Tabella 7.1.4.2:** Risultati dettagliati per l’Analisi su Singola Tabella di Milano

**Commento tabella 7.1.4.2:** con l’aumentare della scala dei dati, i divari prestazionali iniziano a mostrarsi. *PostGIS* è il più efficiente nei calcoli geometrici pesanti (operazione 3.2). Allo stesso tempo, la sua prestazione nell’anti-join (operazione 3.3) degrada significativamente, mentre *DuckDB* e *GeoPandas* mantengono tempi eccellenti, dimostrando una migliore scalabilità per questo specifico compito. Le metriche di sistema misurate evidenziano che *GeoPandas* presenta consumi RAM elevati in tutte e tre le operazioni (dai **352** ai **378 MB**), circa **2.7 volte superiori** a *DuckDB* (**≈140 MB**) e **4.5 volte** rispetto a *PostGIS* (**≈83 MB**), a conferma dei costi computazionali delle strutture dati processate *in-memory*. Nonostante le prestazioni di *PostGIS* nella terza operazione siano peggiorate a causa dell’aumento delle dimensioni del dataset, l’utilizzo della CPU (**16.4%**) rimane il più basso tra gli stack tecnologici. *DuckDB* risulta essere mediamente il più efficiente in tutte le operazioni, tenendo conto sia di prestazioni che di consumi.

Risultati per Roma

| Operazione                                         | Tecnologia     | Tempo (s)    | Risultato             | CPU (%)<br>+<br>RAM (MB)   |
|----------------------------------------------------|----------------|--------------|-----------------------|----------------------------|
| 3.1. Top 10 Aree<br>più Grandi                     | DuckDB Spatial | 0.52         | 10 edifici            | 18.1   140.59              |
|                                                    | PostGIS        | <b>0.38</b>  | 10 edifici            | 41.6   <b>82.58</b>        |
|                                                    | GeoPandas      | 2.63         | 10 edifici            | <b>15.9</b>   786.39       |
| 3.2. Area Totale<br>con Buffer                     | DuckDB Spatial | 6.39         | 298.9 km <sup>2</sup> | 17.6   146.17              |
|                                                    | PostGIS        | <b>2.78</b>  | 299.2 km <sup>2</sup> | 34.4   <b>82.46</b>        |
|                                                    | GeoPandas      | 9.14         | 299.2 km <sup>2</sup> | <b>16.0</b>   884.95       |
| 3.3. Ristoranti<br>NON vicino a<br>Fermate del Bus | DuckDB Spatial | <b>0.081</b> | 2298<br>ristoranti    | 25.2   165.74              |
|                                                    | PostGIS        | 40.34        | 2298<br>ristoranti    | <b>15.8</b>   <b>82.90</b> |
|                                                    | GeoPandas      | 0.13         | 2299<br>ristoranti    | 16.6   884.77              |

**Tabella 7.1.4.3:** Risultati dettagliati per l’Analisi su Singola Tabella di Roma

**Commento tabella 7.1.4.3:** i risultati ottenuti applicando le operazioni su Milano si sono confermati ed anche amplificati sui dataset di Roma. *PostGIS* garantisce il suo primato nei calcoli geometrici (operazioni 3.1 e 3.2), dimostrando un’eccellente scalabilità. Al contrario, la sua prestazione nell’anti-join (operazione 3.3) continua a peggiorare, rendendolo quasi **500 volte più lento** di *DuckDB* e **310 volte più lento** di *GeoPandas*, che si confermano le scelte più sensate per questo tipo di analisi.

Nelle prime due operazioni, *PostGIS* utilizza in modo intensivo la CPU (**41.6%** e **34.4%**) per garantire le sue ottime prestazioni, confermando che il motore *GEOS* scala efficacemente con dataset complessi. Tuttavia, il consumo RAM a picco rimane estremamente contenuto (**≈83 MB**), dimostrando la superiorità architetturale di un sistema di database relazionale.

*GeoPandas*, al contrario, continua a peggiorare nei consumi della RAM, dato che raggiunge gli **885 MB** al picco (quasi **2.5 volte** rispetto a Milano), rivelando il limite della strategia *in-memory* su dataset di grandi dimensioni.

*DuckDB* continua a mantenere un comportamento bilanciato su tutte e tre le operazioni, affermandosi come la tecnologia più affidabile per operazioni analitiche su file di dimensione crescente.

7.1.5. Caso d’uso 4: Join Spaziale Complesso

Questo *caso d’uso* testa il rendimento delle tre tecnologie effettuando operazioni di join spaziale multi-tabella sui dataset precedentemente utilizzati nel terzo *caso d’uso* e su nuovi dataset specifici sempre appartenenti a Pinerolo, Milano e Roma.

Risultati per Pinerolo

| Operazione                                                 | Tecnologia     | Tempo (s)    | Risultato             | CPU (%)<br>+<br>RAM (MB)    |
|------------------------------------------------------------|----------------|--------------|-----------------------|-----------------------------|
| 4.1. Ristoranti per Quartiere                              | DuckDB Spatial | <b>0.004</b> | 13 quartieri          | <b>11.8</b>   245.12        |
|                                                            | PostGIS        | 0.015        | 13 quartieri          | 18.9   226.96               |
|                                                            | GeoPandas      | 0.005        | 13 quartieri          | 12.1   <b>226.04</b>        |
| 4.2. Conteggio Alberi e Lunghezza Strade vicino a Ospedali | DuckDB Spatial | 0.118        | 0 alberi<br>5.0 km    | 20.6   262.96               |
|                                                            | PostGIS        | 0.079        | 0 alberi<br>5.0 km    | <b>13.1</b>   <b>226.99</b> |
|                                                            | GeoPandas      | <b>0.008</b> | 0 alberi<br>5.0 km    | 14.1   228.43               |
| 4.3. Area NON Coperta da Parchi                            | DuckDB Spatial | 0.025        | 49.97 km <sup>2</sup> | 17.3   260.53               |
|                                                            | PostGIS        | 0.006        | 49.97 km <sup>2</sup> | 10.2   <b>227.01</b>        |
|                                                            | GeoPandas      | <b>0.005</b> | 49.97 km <sup>2</sup> | <b>9.5</b>   229.09         |

Tabella 7.1.5.1: Risultati dettagliati per Join Spaziale Complesso di Pinerolo

**Commento tabella 7.1.5.1:** su piccola scala, tutte le tecnologie mostrano tempi di esecuzione molto ridotti. *DuckDB* risulta il più rapido nel join di contenimento (operazione 4.1), mentre *GeoPandas* mostra un netto vantaggio nel join di prossimità (operazione 4.2). Nell’operazione con join spaziali di differenza (operazione 4.3), *GeoPandas* è il più veloce, con *PostGIS* che offre prestazioni quasi identiche, a conferma della robustezza del motore *GEOS* su cui entrambi si basano. *DuckDB* presenta un consumo al picco della RAM leggermente superiore (**≈256 MB**) rispetto a *PostGIS* (**≈227 MB**) e *GeoPandas* (**≈228 MB**). Nonostante ciò, l’utilizzo della CPU rimane competitivo in tutte le operazioni, dimostrando che su piccola scala i tre strumenti sono altamente efficienti.

## Risultati per Milano

| Operazione                                                          | Tecnologia     | Tempo (s)    | Risultato              | CPU (%)<br>+<br>RAM (MB)    |
|---------------------------------------------------------------------|----------------|--------------|------------------------|-----------------------------|
| 4.1. Ristoranti<br>per Quartiere                                    | DuckDB Spatial | 0.097        | 34 quartieri           | 18.3   275.75               |
|                                                                     | PostGIS        | 0.201        | 34 quartieri           | 14.1   <b>230.33</b>        |
|                                                                     | GeoPandas      | <b>0.009</b> | 34 quartieri           | <b>14.0</b>   270.72        |
| 4.2. Conteggio<br>Alberi e<br>Lunghezza Strade<br>vicino a Ospedali | DuckDB Spatial | 0.155        | 234 alberi<br>25.0 km  | 19.5   283.09               |
|                                                                     | PostGIS        | 33.11        | 234 alberi<br>25.0 km  | <b>14.6</b>   <b>230.29</b> |
|                                                                     | GeoPandas      | <b>0.082</b> | 234 alberi<br>25.1 km  | 16.6   273.36               |
| 4.3. Area NON<br>Coperta da Parchi                                  | DuckDB Spatial | 0.198        | 162.98 km <sup>2</sup> | 19.4   283.90               |
|                                                                     | PostGIS        | <b>0.138</b> | 162.98 km <sup>2</sup> | 14.7   <b>227.71</b>        |
|                                                                     | GeoPandas      | 0.423        | 162.98 km <sup>2</sup> | <b>13.7</b>   275.72        |

Tabella 7.1.5.2: Risultati dettagliati per Join Spaziale Complesso di Milano

**Commento tabella 7.1.5.2:** i dati di Milano evidenziano chiaramente i *trade-off*. *GeoPandas* diventa esponenzialmente più veloce nei join di contenimento e di prossimità (operazioni 4.1 e 4.2), risultando **400 volte più rapido** di *PostGIS*, che continua a confermarsi la scelta migliore per operazioni con join spaziali di differenza (operazione 4.3). *DuckDB* dimostra comunque stabilità su tutte e tre le operazioni. Il degrado prestazionale di *PostGIS* nella seconda operazione non è dovuto a un bottleneck dall'utilizzo della CPU oppure dal consumo della RAM, dato che per entrambe le metriche risulta essere il più efficiente (**14.6%** e **≈230 MB**), ma dalla strategia di esecuzione: *PostGIS* impiega un approccio **brute-force** [96] su join di prossimità senza indici pre-calcolati, mentre *GeoPandas* beneficia della funzione specializzata `sjoin_nearest`. *DuckDB* mantiene un consumo della RAM poco più elevato delle altre due tecnologie (**≈283 MB**), accompagnato da un moderato utilizzo della CPU (**19.5%**).

## Risultati per Roma

| Operazione                                                          | Tecnologia     | Tempo (s)    | Risultato              | CPU (%)<br>+<br>RAM (MB)    |
|---------------------------------------------------------------------|----------------|--------------|------------------------|-----------------------------|
| 4.1. Ristoranti<br>per Quartiere                                    | DuckDB Spatial | 4.36         | 52 quartieri           | 20.6   308.42               |
|                                                                     | PostGIS        | 5.32         | 52 quartieri           | 15.5   <b>237.48</b>        |
|                                                                     | GeoPandas      | <b>0.031</b> | 52 quartieri           | <b>13.6</b>   332.57        |
| 4.2. Conteggio<br>Alberi e<br>Lunghezza Strade<br>vicino a Ospedali | DuckDB Spatial | 0.258        | 441 alberi<br>103.7 km | 24.8   310.82               |
|                                                                     | PostGIS        | 186.80       | 441 alberi<br>103.1 km | <b>12.7</b>   <b>229.34</b> |
|                                                                     | GeoPandas      | <b>0.186</b> | 441 alberi<br>103.3 km | 12.9   349.67               |
| 4.3. Area NON<br>Coperta da Parchi                                  | DuckDB Spatial | 0.335        | 1235 km <sup>2</sup>   | 20.4   304.09               |
|                                                                     | PostGIS        | <b>0.236</b> | 1235 km <sup>2</sup>   | <b>12.9</b>   <b>181.34</b> |
|                                                                     | GeoPandas      | 0.784        | 1235 km <sup>2</sup>   | 13.3   350.82               |

Tabella 7.1.5.3: Risultati dettagliati per Join Spaziale Complesso di Roma

**Commento tabella 7.1.5.3:** su scala massima, le differenze sono ancora più grandi. Il vantaggio di *GeoPandas* nei join di contenimento e di prossimità (operazioni 4.1 e 4.2) diventa estremo, dato che risulta **1000 volte più veloce** di *PostGIS*.

*DuckDB* si posiziona come il sistema più bilanciato, avendo ottime prestazioni in quasi tutti gli scenari, mentre *PostGIS* si conferma definitivamente la scelta migliore per i join spaziali di differenza (operazione 4.3).

Nonostante nella seconda operazione *PostGIS* peggiori nelle prestazioni all'aumentare delle dimensioni del dataset, il consumo della RAM rimane stabile a circa **229 MB** e l'utilizzo medio della CPU risulta il più basso tra le tre tecnologie (**12.7%**), dimostrando ancora una volta che il bottleneck è puramente algoritmico.

*GeoPandas* risulta in tutte le operazioni lo strumento che consuma più RAM al picco (in media **345 MB**), ma allo stesso tempo mantiene un utilizzo ridotto della CPU (**13%**). *DuckDB* continua a confermarsi come lo strumento che mantiene il miglior equilibrio tra prestazioni e consumi, anche se in alcuni casi arriva ad utilizzare la CPU il doppio delle altre tecnologie.

### 7.1.6. Caso d'uso 5: Analisi Vettoriale-Raster

Questo *caso d'uso* finale confronta la capacità degli stack tecnologici nell'eseguire analisi di statistiche zonali, combinando dati *vettoriali* con dati *raster*.

| Operazione                    | Tecnologia           | Tempo (s)   | Risultato                    |
|-------------------------------|----------------------|-------------|------------------------------|
| Popolazione Totale per Comune | DuckDB Spatial       | N/A         | N/A                          |
|                               | PostGIS              | <b>2.43</b> | 4M di persone in 1180 comuni |
|                               | Python (rasterstats) | 13.45       | 4M di persone in 1180 comuni |

**Tabella 7.1.6.1:** Confronto prestazioni per Analisi Vettoriale-Raster della popolazione in Piemonte

**Commento tabella 7.1.6.1:** per l'analisi *vettoriale-raster*, *PostGIS* si dimostra superiore allo stack Python che usufruisce della libreria *rasterstats*, poiché risulta essere circa **5.5 volte più veloce**. Ciò suggerisce che, per le operazioni che combinano entrambi i modelli di dati, le funzioni ottimizzate e a basso livello implementate nel database forniscono un vantaggio prestazionale notevole rispetto alle librerie Python di alto livello.

## 7.2. Consigli applicativi

Sulla base dei risultati ottenuti, la seguente tabella riassuntiva fornisce una guida pratica per la scelta della tecnologia più appropriata tra *DuckDB*, *PostGIS* e *GeoPandas* in funzione dello scenario, dei dati, delle specifiche di concorrenza e del tipo di operazioni richieste.

| Scenario                                     | Dati                                   | Concorrenza    | Operazioni tipiche                                                                     | Scelta consigliata                                                                                                                               |
|----------------------------------------------|----------------------------------------|----------------|----------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------|
| Analisi esplorativa / Notebook               | Fino a 2-5 GB<br>Massimo 1-5M features | Singolo utente | Join, buffer, nearest, analisi esplorative, visualizzazioni veloci                     | <b>GeoPandas + DuckDB</b> per I/O veloce su formati non supportati nativamente                                                                   |
| Batch ETL / Preprocessing                    | 5-50M features<br>(flat files)         | Bassa          | Ingestione da formati diversi, filtri complessi, aggregazioni, join tra file           | <b>DuckDB</b> , ottimo per leggere e combinare file GeoParquet con quelli CSV                                                                    |
| Applicazione multi-utente / Dati persistenti | Oltre 10M features con storicizzazione | Alta           | Tutto lo spettro di query spaziali e non, con garanzie di integrità (transazioni ACID) | <b>PostGIS</b> , ideale per backend di servizi web e gestionali                                                                                  |
| Calcoli pesanti                              | Qualsiasi scala                        | Singolo utente | Calcoli geometrici molto intensivi, come union o difference su milioni di geometrie    | <b>PostGIS</b> , il motore GEOS mostra superiorità su calcoli complessi                                                                          |
| Analisi Vettoriale-Raster                    | Griglie medio-grosse                   | Bassa/Media    | Statistiche zonali, resampling, clip di raster basati su vettori                       | <b>PostGIS Raster</b> per analisi complesse; stack Python ( <b>raSTERio</b> + <b>GeoPandas</b> ) per esplorazioni rapide e maggiore flessibilità |
| API / Servizi GIS                            | Dati indicizzati medio-grossi          | Media/Alta     | Interrogazioni spaziali specifiche e parametriche, buffer dinamici                     | <b>PostGIS</b> , performance garantite da indici e gestione della concorrenza; <b>DuckDB</b> per lavori offline                                  |

Tabella 7.2.0.1: guida alla scelta della tecnologia geospaziale in funzione dello scenario di utilizzo.

## 7.3. Scoperte significative

L'analisi comparativa ha prodotto una serie di scoperte significative, sia quantitativamente che qualitativamente, sui *trade-off* tra questi sistemi.

Queste scoperte sono le seguenti:

- **Criticità nelle estrazioni:** durante i primi benchmark, è emersa una significativa discrepanza nel numero di edifici estratti per la stessa area (Milano) tra *GeoPandas* ( $\approx 60$  mila) e gli altri due strumenti ( $\approx 62$  mila). Una fase successiva del lavoro è stata investigare se la causa risiede negli algoritmi di intersezione usati dalla tecnologia (*clip* e *intersects*) o nella gestione delle geometrie invalide.
  - Dopo aver effettuato nuovi test e modificato lo script precedentemente utilizzato, si è giunti alla conclusione che il problema fosse nel **non** estrarre dati anche per gli elementi di tipo *relation*, visto che sono dei *multipoligoni* formati da, in questo caso, elementi di tipo *way*. Effettuando l'estrazione di entrambi i tipi di elementi, processando le *relations* e utilizzando le *ways* messe in cache per assemblare correttamente le geometrie complesse (*multipoligoni*), si sono ottenuti risultati molto simili tra tutte e tre le tecnologie:
    - \* Edifici estratti con *DuckDB*: **62133**
    - \* Edifici estratti con *PostGIS*: **62127**
    - \* Edifici estratti con *GeoPandas*: **62075**
  - Da come si può notare, il valore non è del tutto identico, ma sicuramente è un grande risultato rispetto ai valori ottenuti in precedenza. Ciò dimostra che l'accuratezza dei risultati ottenuti con strumenti di basso livello è direttamente dipendente da una profonda comprensione del modello dati sottostante.
  - Successivamente si potrebbero fare ulteriori analisi, come confrontare gli *id* degli edifici, così da capire quali siano quelli mancanti e se sia presente un valido motivo per cui non siano stati estratti tramite *pyosmium*.
- **Diagnosi dei bottleneck di prestazione:** l'implementazione della metodologia di test "*cold start/hot cache*" si è rivelata cruciale, visto che ha permesso di diagnosticare con precisione il bottleneck presente per ogni tipo di tecnologia:
  - Le operazioni di lettura di file di grandi dimensioni sono risultate limitate dalla velocità del disco (un fenomeno noto come **I/O-bound** [100]), dove la cache del sistema operativo offre miglioramenti nelle esecuzioni successive (quasi 6 volte più veloce).
  - Le query eseguite su *PostGIS* mostrano un comportamento simile, dove la prima esecuzione è più lenta perché deve caricare i dati dal disco nella sua cache, con notevoli velocizzazioni nelle query successive, dimostrando che *PostGIS* sia **Cache-bound**.
  - Le operazioni di calcolo intensivo sono risultate limitate dalla CPU. Viene reso evidente con *GeoPandas*, dato che i tempi di esecuzione in *cold start* e in *hot cache* sono praticamente identici, e in quasi tutte le operazioni dei

casi d'uso 3 e 4, dove anche per *DuckDB* e *PostGIS* la cache non porta alcun beneficio significativo essendo che il costo del calcolo domina su quello della lettura dei dati. Questo accade a causa del bottleneck **CPU-bound** [95].

- Naturalmente, anche la RAM fa da fattore limitante, visto che i test vengono effettuati su due banchi da 8 GB ciascuno, non permettendo di gestire enormi quantità di dati tutti assieme, anche se il principale ostacolo è la capacità di calcolo della CPU.
- **Impatto della qualità e della coerenza dei dati:** una delle scoperte più importanti non riguarda la velocità di esecuzione, ma la causa radice della correttezza e della coerenza dei risultati.
  - Inizialmente, i benchmark dei casi d'uso 3 e 4 avevano rivelato significative discrepanze numeriche. *PostGIS* e *GeoPandas* (entrambi basati sulla libreria geometrica *GEOS*) producevano risultati quasi identici, stabilendo un solido riferimento di correttezza. *DuckDB*, al contrario, si comportava come un “outlier”, calcolando aree errate (operazioni 3.2 e 4.3) e restituendo un numero di feature diverso nei join di prossimità (operazione 4.2).
  - L'ipotesi iniziale puntava a una minore robustezza delle funzioni di *DuckDB* (es.: `ST_Union_Agg`) o ad un'implementazione non standardizzata. Tuttavia, un processo di debugging mirato ha rivelato la vera causa: un disallineamento del metadato del *CRS* (*Coordinate Reference System*).
  - L'indagine ha dimostrato che i file *GeoParquet* di input utilizzavano l'identificatore `OGC:CRS84`, mentre le query *DuckDB* specificavano erroneamente `EPSG:4326` come *CRS* di partenza nella funzione `ST_Transform`. Sebbene entrambi i codici si riferiscano a *WGS 84*, l'implementazione di *DuckDB* si è rivelata estremamente rigorosa nel richiedere una corrispondenza esatta, a differenza delle tecnologie *GEOS-based*, che gestivano l'ambiguità in maniera totalmente trasparente.
  - La risoluzione di questo problema, ottenuta semplicemente correggendo il *CRS* di partenza nelle query SQL, ha avuto un impatto trasformativo:
    - \* Tutte le discrepanze numeriche sono state risolte. I risultati di *DuckDB* per le operazioni 3.2, 4.2 e 4.3 si sono allineati con il riferimento stabilito da *PostGIS* e *GeoPandas*.
    - \* La robustezza di *DuckDB* è stata confermata. Il problema nell'operazione 4.3 (Area non Coperta da Parchi) non era un fallimento della funzione `ST_Union_Agg` nel gestire geometrie complesse, ma una conseguenza della trasformazione di coordinate errata che generava dati invalidi.
  - Questa scoperta sposta la conclusione finale: il *trade-off* non è tra la velocità di *DuckDB* e la robustezza delle architetture *GEOS-based*, ma tra la maturità e la flessibilità di uno standard consolidato (*GEOS*) e le prestazioni elevate di un sistema più recente (*DuckDB*), che però richiede una configurazione più precisa e rigorosa.

- **Gerarchia delle prestazioni per operazione (trade-off architetturali):** non esiste una tecnologia completamente migliore rispetto le altre due, però si possono sviluppare delle gerarchie a seconda del compito da svolgere:
  - Per l'ingestione di file, *DuckDB* e *GeoPandas* sono più veloci dei tool CLI per database, come *shp2pgsql* e *raster2pgsql*. Strumenti come *quackosm* e *pyosmium* sono più rapidi ad estrarre specifici sottoinsiemi, ma possono comportare maggiori consumi di memoria.
  - Nei calcoli pesanti, *PostGIS* si è mostrato il più performante, gestendo calcoli su milioni di geometrie molto più velocemente sia di *DuckDB* che di *GeoPandas*.
  - *GeoPandas*, grazie alla sua funzione `sjoin_nearest`, riesce ad effettuare join di prossimità in tempi straordinariamente più veloci rispetto alle altre due tecnologie, completando l'operazione in frazioni di secondo contro le decine di secondi impiegate da *DuckDB* e *PostGIS*. Tuttavia, per gli anti-join di prossimità, si è osservato che lo stesso algoritmo produceva un conteggio di risultati diverso rispetto all'approccio basato su `ST_DWithin` dei database. Per risolvere questa discrepanza, è stata implementata una metodologia più rigorosa (`buffer + sjoin`) che, pur con un moderato costo prestazionale, ha garantito la piena coerenza numerica. Questa scoperta evidenzia un *trade-off* non tra tecnologie, ma all'interno dello stesso strumento: la scelta dell'algoritmo ottimale dipende dalla specifica natura dell'operazione spaziale.
  - Nell'analisi *vettoriale-raster*, *PostGIS* ha dimostrato di essere nettamente superiore, visto che risulta essere quasi 6 volte più veloce dello stack Python per il calcolo di statistiche zonali.
  - *DuckDB* risulta un ottimo strumento nell'esecuzione di ogni tipo di operazione, nonostante sia raramente il più rapido tra le tre tecnologie, mantenendo comunque tempi di risoluzione bassi ed, in certe occasioni, estremamente simili a quelli dello strumento più veloce. L'unico "difetto" di *DuckDB* si pone, naturalmente, quando si ha a che fare con dati *raster*, a causa della sua mancata compatibilità.
- **Gestione centrale dei CRS:** il debugging ripetuto ha confermato che una gestione attenta dei *Coordinate Reference Systems (CRS)* è il prerequisito fondamentale per ottenere risultati corretti. L'esecuzione di calcoli metrici su dati in coordinate geografiche inesatte è un errore molto comune, che è stato notato spesso con la sbagliata implementazione di query con funzioni `ST_DWithin`, ma allo stesso tempo è anche un errore molto grave, dato che i risultati ottenuti appaiono completamente discostati dai valori reali.



## 8. Conclusioni

Questo progetto di tesi ha contribuito allo studio comparativo degli strumenti attuali per l'analisi geospaziale, confrontando soluzioni consolidate come *PostGIS* con tecnologie più recenti, quali *DuckDB* e l'ecosistema di *GeoPandas*. Piuttosto che identificare una tecnologia superiore sotto ogni aspetto rispetto alle altre due, la ricerca si è concentrata sull'analisi dei compromessi tra prestazioni, stabilità e precisione numerica attraverso i *casi d'uso* pratici che sono stati sviluppati appositamente.

I risultati dei benchmark hanno rivelato uno scenario piuttosto complesso, dato che ogni tecnologia presenta caratteristiche distintive: *GeoPandas* si distingue nei join di prossimità quando i dati possono essere elaborati interamente in memoria, *PostGIS* conferma la sua posizione di riferimento per robustezza e prestazioni nei calcoli geometrici complessi su dataset di grandi dimensioni, mentre *DuckDB* si afferma come una soluzione bilanciata e versatile, mostrando particolare efficacia come strumento d'uso generale di fronte a qualsiasi tipo di lavoro.

L'aspetto più significativo emerso dall'analisi riguarda, tuttavia, la relazione tra velocità e correttezza dei risultati. L'indagine sulle discrepanze numeriche riscontrate in *DuckDB* ha evidenziato che l'affidabilità della tecnologia dipendeva non tanto da limitazioni algoritmiche, quanto da una gestione accurata dei metadati relativi ai sistemi di riferimento (*CRS*). Questo solleva una questione rilevante: mentre gli strumenti più recenti offrono prestazioni superiori, grazie soprattutto all'assenza di un server di database da gestire, il loro utilizzo efficace richiede competenze più approfondite nella validazione e gestione dei dati rispetto agli ambienti consolidati. In definitiva, lo studio dimostra che la scelta di uno stack tecnologico costituisce una decisione che va oltre gli aspetti puramente tecnici, richiedendo uno studio ragionato per ottenere l'equilibrio tra necessità prestazionali, garanzie di affidabilità e solidità dell'ecosistema di sviluppo, soprattutto in base alla tipologia di lavoro che si andrà a svolgere.

### 8.1. Limitazioni dello studio

Lo studio effettuato con questo progetto ha sicuramente avuto delle limitazioni a causa dell'hardware del sistema utilizzato, poiché le problematiche riscontrate, come descritto nel capitolo precedente, riguardavano frequentemente diverse tipologie di *bottleneck*.

Il sistema è composto dalle seguenti componenti hardware:

- **CPU** (2019) con 6 core, 12 thread e frequenza base di 3.6 GHz [88].
- **RAM** (2019) DDR4 da 8 GB per due banchi, con velocità di memoria di 3600 MHz e latenza CAS 16 [64].

- **Memoria SSD primaria** (2019) da 1 TB in formato M.2 2280, con velocità di lettura 2200 MB/s e scrittura 2000 MB/s [90].
- **Memoria SSD secondaria** (2018) da 1TB in formato 2.5", con velocità di lettura 560 MB/s e scrittura 510 MB/s [89].

Come si può notare, le componenti del sistema non sono molto recenti e, di conseguenza, non permettono prestazioni ottimali come quelle di ultima generazione. Ad esempio, le CPU odierne possiedono più core e thread, ma consumano nettamente meno, garantendo prestazioni a regime migliori.

Inoltre, bisogna anche parlare di un'importante limitazione dal lato software. *PostGIS* sfrutta l'ambiente del suo server di database esterno, mentre *DuckDB* e *GeoPandas* usufruiscono dell'ecosistema Python. Per questo motivo, alcune librerie dei due ambienti sono aggiornate a versioni differenti:

| Tecnologia         | Versione GEOS | Versione PROJ |
|--------------------|---------------|---------------|
| PostGIS            | 3.10.2        | 8.2.1         |
| DuckDB e GeoPandas | 3.11.1        | 9.1.0         |

Queste differenze tra le versioni delle librerie non comportano un problema vero e proprio, visto che i dati ottenuti dallo svolgimento di ogni *caso d'uso* risultano numericamente simili, ma mostrano come la riproducibilità di determinate operazioni dipenda dallo strumento utilizzato e anche dal suo ambiente di sviluppo.

## 8.2. Sviluppi futuri

Sapendo che la ricerca è stata condizionata da determinati fattori, non direttamente risolvibili dall'utente e dal sistema utilizzato, la si potrebbe estendere e approfondire seguendo specifiche direzioni:

- **Analisi hardware:** replicare i benchmark su sistemi con differenti configurazioni hardware (es.: memorie NVMe M.2 con maggiore velocità di lettura e scrittura, CPU con più core, maggiori quantità di RAM) per analizzare i cambiamenti dei bottleneck prestazionali.
- **Inclusione di nuove tecnologie:** ampliare l'insieme di tecnologie su cui effettuare l'analisi comparativa. Ad esempio, si potrebbero includere database analitici che stanno sviluppando estensioni spaziali, come ClickHouse [8, 9].
- **Estensione dei casi d'uso:** introdurre nuovi scenari focalizzati su operazioni geospaziali complesse non trattate, come il routing (calcolo di percorsi ottimali su reti stradali) o l'analisi di dati 3D.
- **Aggiunta di dataset differenti:** scaricare dataset di dimensioni e tipologie diverse rispetto a quelle dei dataset usati nel progetto. Ad esempio, si potrebbero usare file con dati vettoriali di città molto più grandi, come Houston o Chongqing.

- **Studio longitudinale:** valutare l'evoluzione delle prestazioni e della robustezza delle librerie utilizzate una volta rilasciate nuove versioni, per monitorare i comportamenti futuri degli strumenti analizzati.



# Bibliografia

- [1] A. Anwar. Hands-on Introduction to DuckDB (2025). *To Data & Beyond*, Consultato: settembre 2025. Articolo blog: <https://youssefh.substack.com/p/hands-on-introduction-to-duckdb>.
- [2] The SQLAlchemy Authors and Contributors. SQLAlchemy - SQL Toolkit and Object Relational Mapper (2025). Documentazione: <https://docs.sqlalchemy.org/en/20/intro.html>, Consultato: settembre 2025.
- [3] Amazon Web Services (AWS). Archiviazione di oggetti cloud - Amazon S3 (2025). Documentazione: <https://aws.amazon.com/it/s3/>, Consultato: settembre 2025.
- [4] Amazon Web Services (AWS). Cos'è l'ETL (Estrazione, trasformazione, caricamento)? (2025). Documentazione: <https://aws.amazon.com/what-is/etl/>, Consultato: settembre 2025.
- [5] A. Berchet and GeoAlchemy2 Developers. geoalchemy/geoalchemy2: 0.18.0 (0.18.0) (2025). *Zenodo*, Consultato: settembre 2025. Documentazione: <http://geoalchemy-2.readthedocs.io/en/latest/>. doi: <https://doi.org/10.5281/zenodo.16267005>.
- [6] G. Boeing. Modeling and Analyzing Urban Networks and Amenities With OSMnx (2025). *Geographical Analysis*, Consultato: settembre 2025. Articolo: <https://onlinelibrary.wiley.com/doi/10.1111/gean.70009>. doi: <http://doi.org/10.1111/gean.70009>.
- [7] A. Carioli, M. Schiavina, S. Freire, and K. MacManus. GHS-POP R2023A - GHS population grid multitemporal (2023). *European Commission, Joint Research Centre (JRC)*, Consultato: settembre 2025. Documentazione: [https://ghsl.jrc.ec.europa.eu/ghs\\_pop2023.php](https://ghsl.jrc.ec.europa.eu/ghs_pop2023.php). PID: <http://data.europa.eu/89h/2ff68a52-5b5b-4a22-8f40-c41da8332cfe>. doi: <https://doi.org/10.2905/2FF68A52-5B5B-4A22-8F40-C41DA8332CFE>.
- [8] Inc. ClickHouse. Geo Functions | ClickHouse Docs (2025). Documentazione: <https://clickhouse.com/docs/sql-reference/functions/geo>, Consultato: settembre 2025.
- [9] Inc. ClickHouse. What is ClickHouse? | ClickHouse Docs (2025). Documentazione: <https://clickhouse.com/docs/intro>, Consultato: settembre 2025.

- [10] OutSystems Community. Optimize Data For Your Use Cases Using Hot Cache (2025). Documentazione: [https://success.outsystems.com/documentation/11/app\\_architecture/optimize\\_data\\_for\\_your\\_use\\_cases\\_using\\_hot\\_cache/](https://success.outsystems.com/documentation/11/app_architecture/optimize_data_for_your_use_cases_using_hot_cache/), Consultato: settembre 2025.
- [11] Stack Overflow Community. 'geopandas' tag wiki (2014). Documentazione: <https://stackoverflow.com/tags/geopandas/info>, Consultato: settembre 2025.
- [12] GDAL/OGR Contributors. GDAL/OGR Geospatial Data Abstraction software Library (2025). *Open Source Geospatial Foundation*, Consultato: settembre 2025. Documentazione: <https://gdal.org>. doi: <https://doi.org/10.5281/zenodo.17098582>.
- [13] GeoJSON Contributors. GeoJSON (2025). Pagina web: <https://geojson.org/>, Consultato: settembre 2025.
- [14] GeoParquet Open Source Contributors, Element 84, CARTO, Foursquare, GeoPandas, GeoTrellis, Microsoft, Planet, Apache Sedona, GDAL, Voltron Data, Wherobots, Google BigQuery, Databricks, Amazon Redshift, Snowflake, and Overture Maps Foundation. GeoParquet - Geospatial data in Parquet (2025). Documentazione: <https://geoparquet.org/>, Consultato: settembre 2025.
- [15] GEOS Contributors. GEOS computational geometry library (2024). *Open Source Geospatial Foundation*, Consultato: settembre 2025. Documentazione: <https://libgeos.org/>. doi: <https://doi.org/10.5281/zenodo.11396894>.
- [16] GEOS Contributors. PostGIS - OSGeo (2025). Documentazione: <https://www.osgeo.org/projects/postgis/>, Consultato: settembre 2025.
- [17] OpenStreetMap Wiki Contributors. Downloading data - OpenStreetMap Wiki (2025). Pagina web: [https://wiki.openstreetmap.org/wiki/Downloading\\_data](https://wiki.openstreetmap.org/wiki/Downloading_data), Consultato: settembre 2025.
- [18] OpenStreetMap Wiki Contributors. Elements - OpenStreetMap Wiki (2025). Pagina web: <https://wiki.openstreetmap.org/wiki/Elements>, Consultato: settembre 2025.
- [19] OpenStreetMap Wiki Contributors. Node - OpenStreetMap Wiki (2025). Pagina web: <https://wiki.openstreetmap.org/wiki/Node>, Consultato: settembre 2025.
- [20] OpenStreetMap Wiki Contributors. OpenStreetMap - The Free Wiki World Map (2025). Sito web: [https://wiki.openstreetmap.org/wiki/Main\\_Page](https://wiki.openstreetmap.org/wiki/Main_Page), Consultato: settembre 2025.
- [21] OpenStreetMap Wiki Contributors. PBF Format - OpenStreetMap Wiki (2025). Pagina web: [https://wiki.openstreetmap.org/wiki/PBF\\_Format](https://wiki.openstreetmap.org/wiki/PBF_Format), Consultato: settembre 2025.

- [22] OpenStreetMap Wiki Contributors. Relation - OpenStreetMap Wiki (2025). Pagina web: <https://wiki.openstreetmap.org/wiki/Relation>, Consultato: settembre 2025.
- [23] OpenStreetMap Wiki Contributors. Tags - OpenStreetMap Wiki (2025). Pagina web: <https://wiki.openstreetmap.org/wiki/Tags>, Consultato: settembre 2025.
- [24] OpenStreetMap Wiki Contributors. Way - OpenStreetMap Wiki (2025). Pagina web: <https://wiki.openstreetmap.org/wiki/Way>, Consultato: settembre 2025.
- [25] PROJ Contributors. PROJ coordinate transformation software library (2025). *Open Source Geospatial Foundation*, Consultato: settembre 2025. Documentazione: <https://proj.org/>. doi: <https://doi.org/10.5281/zenodo.5884394>.
- [26] IBM Corporation. Cosa sono i dati geospaziali? | IBM (2020). Documentazione: <https://www.ibm.com/it-it/topics/geospatial-data>, Consultato: settembre 2025.
- [27] Refsnes Data. SQL Joins - W3schools (2024). Pagina web: [https://www.w3schools.com/sql/sql\\_join.asp](https://www.w3schools.com/sql/sql_join.asp), Consultato: settembre 2025.
- [28] Esri Developer. Cos'è il GIS? | Tecnologia di mappatura del sistema informativo geografico (2025). Documentazione: <https://www.esri.com/it-it/what-is-gis/overview>, Consultato: settembre 2025.
- [29] Esri Developer. GeoParquet | ArcGIS GeoAnalytics Engine | Esri Developer (2025). Documentazione: <https://developers.arcgis.com/geoanalytics/data/data-sources/geoparquet/>, Consultato: settembre 2025.
- [30] Esri Developer. Shapefile - Portal for ArcGIS (2025). Documentazione: <https://enterprise.arcgis.com/it/portal/11.2/use/shapefiles.htm>, Consultato: settembre 2025.
- [31] Esri Developer. What is an SRID? - ArcMap (2021). Documentazione: <https://desktop.arcgis.com/en/arcmap/latest/manage-data/using-sql-with-gdbs/what-is-an-srid.htm>, Consultato: settembre 2025.
- [32] GeoPandas Developers. Ecosystem (2025). Documentazione: <https://geopandas.org/en/stable/community/ecosystem.html>, Consultato: settembre 2025.
- [33] GeoPandas Developers. GeoPandas Documentation (2025). Documentazione: <https://geopandas.org/en/stable/>, Consultato: settembre 2025.
- [34] GeoPandas Developers. Introduction to GeoPandas (2024). Documentazione: [https://geopandas.org/en/stable/getting\\_started/introduction.html](https://geopandas.org/en/stable/getting_started/introduction.html), Consultato: settembre 2025.

- [35] GeoPandas Developers. Migration from PyGEOS geometry backend to Shapely 2.0 (2025). Documentazione: [https://geopandas.org/en/stable/docs/user\\_guide/pygeos\\_to\\_shapely.html](https://geopandas.org/en/stable/docs/user_guide/pygeos_to_shapely.html), Consultato: settembre 2025.
- [36] GeoPandas Developers. Reading and writing files (2025). Documentazione: [https://geopandas.org/en/stable/docs/user\\_guide/io.html](https://geopandas.org/en/stable/docs/user_guide/io.html), Consultato: settembre 2025.
- [37] GeoPandas Developers. Spatial indexing (2025). Documentazione: [https://geopandas.org/en/stable/docs/user\\_guide/spatial\\_indexing.html](https://geopandas.org/en/stable/docs/user_guide/spatial_indexing.html), Consultato: settembre 2025.
- [38] ISTAT | Istituto Nazionale di Statistica. Confini delle unità amministrative a fini statistici (2025). Documentazione: <https://www.istat.it/notizia/confini-delle-unita-amministrative-a-fini-statistici-al-1-gennaio-2018-2/>, Consultato: settembre 2025.
- [39] NASA Earth. GeoTIFF | NASA Earthdata (2019). Documentazione: <https://www.earthdata.nasa.gov/about/esdis/esco/standards-practices/geotiff>, Consultato: settembre 2025.
- [40] DuckDB Foundation. DuckDB - An in-process SQL OLAP database management system (2025). Sito web: <https://duckdb.org/>, Consultato: settembre 2025.
- [41] DuckDB Foundation. GDAL Integration (2025). Documentazione: [https://duckdb.org/docs/stable/core\\_extensions/spatial/gdal.html](https://duckdb.org/docs/stable/core_extensions/spatial/gdal.html), Consultato: settembre 2025.
- [42] DuckDB Foundation. https Extension for HTTP and S3 Support (2025). Documentazione: [https://duckdb.org/docs/stable/core\\_extensions/https/overview](https://duckdb.org/docs/stable/core_extensions/https/overview), Consultato: settembre 2025.
- [43] DuckDB Foundation. HTTP(S) Support - Partial Reading (2025). Documentazione: [https://duckdb.org/docs/stable/core\\_extensions/https/https#partial-reading](https://duckdb.org/docs/stable/core_extensions/https/https#partial-reading), Consultato: settembre 2025.
- [44] DuckDB Foundation. Why DuckDB (2025). Documentazione: [https://duckdb.org/why\\_duckdb](https://duckdb.org/why_duckdb), Consultato: settembre 2025.
- [45] M. Gabrielsson. PostGEESE? Introducing The DuckDB Spatial Extension (2023). *DuckDB Foundation*, Consultato: settembre 2025. Articolo blog: <https://duckdb.org/2023/04/28/spatial.html>.
- [46] Geofabrik. Geofabrik Download Server - Download OpenStreetMap data for this region: Italy (2025). Documentazione: <https://download.geofabrik.de/europe/italy.html>, Consultato: settembre 2025.
- [47] Geofabrik. Geofabrik Download Server - OpenStreetMap Data Extracts (2025). Sito web: <https://download.geofabrik.de/>, Consultato: settembre 2025.

- [48] S. Gillies. Fiona: access to simple geospatial feature data (2024). Documentazione: <https://fiona.readthedocs.io/en/stable/>, Consultato: settembre 2025.
- [49] S. Gillies. Rasterio: access to geospatial raster data (2018). Documentazione: <https://rasterio.readthedocs.io/en/stable/>, Consultato: settembre 2025.
- [50] S. Gillies. The Fiona User Manual (2024). Documentazione: <https://fiona.readthedocs.io/en/stable/manual.html#the-fiona-user-manual>, Consultato: settembre 2025.
- [51] S. Gillies and Shapely Contributors. Shapely - Manipulation and analysis of geometric objects (2025). Documentazione: <https://shapely.readthedocs.io/>, Consultato: settembre 2025.
- [52] PostgreSQL Global Development Group. GiST Indexes - PostgreSQL 16 Documentation (2025). Documentazione: <https://www.postgresql.org/docs/current/gist.html>, Consultato: settembre 2025.
- [53] The PostGIS Development Group. Data Management - OGC Geometry (2025). Documentazione: [https://postgis.net/docs/manual-dev/it/using\\_postgis\\_dbmanagement.html](https://postgis.net/docs/manual-dev/it/using_postgis_dbmanagement.html), Consultato: settembre 2025.
- [54] The PostGIS Development Group. Data Management - Spatial Indexes (2025). Documentazione: [https://postgis.net/docs/manual-dev/it/using\\_postgis\\_dbmanagement.html#build-indexes](https://postgis.net/docs/manual-dev/it/using_postgis_dbmanagement.html#build-indexes), Consultato: settembre 2025.
- [55] The PostGIS Development Group. Dati raster: gestione, interrogazione e applicazioni (2025). Documentazione: [https://postgis.net/docs/manual-dev/it/using\\_raster\\_dataman.html#RT\\_Raster\\_Loader](https://postgis.net/docs/manual-dev/it/using_raster_dataman.html#RT_Raster_Loader), Consultato: settembre 2025.
- [56] The PostGIS Development Group. `getfaceedges_returntype` (2025). Documentazione: [https://postgis.net/docs/manual-dev/it/getfaceedges\\_returntype.html](https://postgis.net/docs/manual-dev/it/getfaceedges_returntype.html), Consultato: settembre 2025.
- [57] The PostGIS Development Group. Indice delle funzioni speciali di PostGIS (2025). Documentazione: [https://postgis.net/docs/manual-dev/it/PostGIS\\_Special\\_Functions\\_Index.html](https://postgis.net/docs/manual-dev/it/PostGIS_Special_Functions_Index.html), Consultato: settembre 2025.
- [58] The PostGIS Development Group. PostGIS - Spatial and Geographic Objects for PostgreSQL (2025). Sito web: <https://postgis.net/>, Consultato: settembre 2025.
- [59] The PostGIS Development Group. `raster2pgsql` - Using `raster2pgsql` to load rasters (2025). Documentazione: [https://postgis.net/docs/using\\_raster\\_dataman.html](https://postgis.net/docs/using_raster_dataman.html), Consultato: settembre 2025.
- [60] The PostGIS Development Group. Riferimento raster (2025). Documentazione: [https://postgis.net/docs/manual-dev/it/RT\\_reference.html](https://postgis.net/docs/manual-dev/it/RT_reference.html), Consultato: settembre 2025.

- [61] The PostGIS Development Group. shp2pgsql - Using the Shapefile Loader (2025). Documentazione: [https://postgis.net/docs/manual-dev/it/using\\_postgis\\_dbmanagement.html#shp2pgsql\\_usage](https://postgis.net/docs/manual-dev/it/using_postgis_dbmanagement.html#shp2pgsql_usage), Consultato: settembre 2025.
- [62] The PostGIS Development Group. Topologia (2025). Documentazione: <https://postgis.net/docs/manual-dev/it/Topology.html>, Consultato: settembre 2025.
- [63] The PostgreSQL Global Development Group. PostgreSQL: The World's Most Advanced Open Source Relational Database (2025). Sito web: <https://www.postgresql.org/>, Consultato: settembre 2025.
- [64] H. Hagedoorn. Ballistix Elite 3600 MHz 16GB Dual Channel DDR4 review (2019). *Guru3D*, Consultato: settembre 2025. Recensione hardware: <https://www.guru3d.com/review/ballistix-elite-3600-mhz-16gb-dual-channel-ddr4-review/>.
- [65] S. Hoffmann and Open Source Contributors. PyOsmium - Python bindings to Osmium Library (2025). Documentazione: <https://osmcode.org/pyosmium/>, Consultato: settembre 2025.
- [66] P. Holanda and J. Keane. DuckDB Quacks Arrow: A Zero-Copy Data Integration between Apache Arrow and DuckDB (2021). *DuckDB Foundation*, Consultato: settembre 2025. Articolo blog: <https://duckdb.org/2021/12/03/duck-arrow.html>.
- [67] Fivetran Inc. How to choose between a columnar database vs. row database (2023). *Fivetran*, Consultato: settembre 2025. Articolo informativo: <https://www.fivetran.com/learn/columnar-database-vs-row-database>.
- [68] European Union's Copernicus Land Monitoring Service information. CORINE Land Cover 2018 (raster 100 m), Europe, 6-yearly - version 2020\_20u1 (2020). *EEA geospatial data catalogue*, Consultato: settembre 2025. Documentazione: <https://sdi.eea.europa.eu/catalogue/copernicus/api/records/960998c1-1870-4e82-8051-6485205ebbac?language=all>. doi: <https://doi.org/10.2909/960998c1-1870-4e82-8051-6485205ebbac>.
- [69] R. Kumar. What is PostgreSQL - Introduction (2023). *GeeksforGeeks*, Consultato: settembre 2025. Pagina web: <https://www.geeksforgeeks.org/postgresql/what-is-postgresql-introduction/>.
- [70] SQLite Consortium members. Appropriate Uses For SQLite (2025). Documentazione: <https://sqlite.org/whentouse.html>, Consultato: settembre 2025.
- [71] MotherDuck. What is DuckDB? (2024). Documentazione: <https://motherduck.com/learn-more/what-is-duckdb/>, Consultato: settembre 2025.
- [72] H. Mühleisen and M. Raasveldt. DuckDB: an Embeddable Analytical Database (2019). *SIGMOD*, Consultato: settembre 2025. Articolo scientifico: <https://ir.cwi.nl/pub/28800/28800.pdf>.

- [73] osm2pgsql Open Source Developers. osm2pgsql - OpenStreetMap data to PostGIS converter (2025). Documentazione: <https://osm2pgsql.org/>, Consultato: settembre 2025.
- [74] M. Perry. rasterstats - documentation (2015). Documentazione: <https://pythonhosted.org/rasterstats/index.html>, Consultato: settembre 2025.
- [75] K. Raczycki and 'kraina-ai' Development Team. QuackOSM: an open-source Python and CLI tool for reading OpenStreetMap PBF files using DuckDB (2025). Repository GitHub: <https://github.com/kraina-ai/quackosm>, Consultato: settembre 2025.
- [76] N. Richardson, I. Cook, N. Crane, D. Dunnington, R. François, J. Keane, D. Moldovan-Grünfeld, J. Ooms, and J. Wujciak-Jens. Apache Arrow - Apache Arrow v21.0.0 (2025). *Apache Software Foundation*, Consultato: settembre 2025. Documentazione: <https://arrow.apache.org/docs/>. Repository GitHub: <https://github.com/apache/arrow/>.
- [77] P. Sexton. What is a geographic information system (GIS)? | U.S. Geological Survey (2025). Sito web: <https://www.usgs.gov/faqs/what-geographic-information-system-gis>, Consultato: settembre 2025.
- [78] Alan D. Snow and Corteva Agriscience. corteva/rioxarray: 0.19.0 Release (0.19.0) (2023). *Zenodo*, Consultato: settembre 2025. Documentazione: <https://corteva.github.io/rioxarray/stable/>. doi: <https://doi.org/10.5281/zenodo.15257348>.
- [79] Pandas Development Team and Inc. NumFOCUS. pandas - powerful Python data analysis toolkit (2025). Documentazione: <https://pandas.pydata.org/>, Consultato: settembre 2025.
- [80] QGIS Development Team. Attributi nei dati vettoriali - Documentazione QGIS (2025). Documentazione: [https://docs.qgis.org/3.40/it/docs/gentle\\_gis\\_introduction/vector\\_attribute\\_data.html#attributes-in-detail](https://docs.qgis.org/3.40/it/docs/gentle_gis_introduction/vector_attribute_data.html#attributes-in-detail), Consultato: settembre 2025.
- [81] QGIS Development Team. Dati raster - Documentazione QGIS (2025). Documentazione: [https://docs.qgis.org/3.40/it/docs/gentle\\_gis\\_introduction/raster\\_data.html](https://docs.qgis.org/3.40/it/docs/gentle_gis_introduction/raster_data.html), Consultato: settembre 2025.
- [82] QGIS Development Team. Dati raster (risoluzione spaziale) - Documentazione QGIS (2025). Documentazione: [https://docs.qgis.org/3.40/it/docs/gentle\\_gis\\_introduction/raster\\_data.html#spatial-resolution](https://docs.qgis.org/3.40/it/docs/gentle_gis_introduction/raster_data.html#spatial-resolution), Consultato: settembre 2025.
- [83] QGIS Development Team. Dati vettoriali - Documentazione QGIS (2025). Documentazione: [https://docs.qgis.org/3.40/it/docs/gentle\\_gis\\_introduction/vector\\_data.html#](https://docs.qgis.org/3.40/it/docs/gentle_gis_introduction/vector_data.html#), Consultato: settembre 2025.
- [84] QGIS Development Team. Dati vettoriali (elementi poligonali) - Documentazione QGIS (2025). Documentazione: [https://docs.qgis.org/3.40/it/docs/gentle\\_gis\\_introduction/vector\\_data.html#polygon-features-in-detail](https://docs.qgis.org/3.40/it/docs/gentle_gis_introduction/vector_data.html#polygon-features-in-detail), Consultato: settembre 2025.

- [85] QGIS Development Team. Dati vettoriali (elementi polilinea) - Documentazione QGIS (2025). Documentazione: [https://docs.qgis.org/3.40/it/docs/gentle\\_gis\\_introduction/vector\\_data.html#polyline-features-in-detail](https://docs.qgis.org/3.40/it/docs/gentle_gis_introduction/vector_data.html#polyline-features-in-detail), Consultato: settembre 2025.
- [86] QGIS Development Team. Dati vettoriali (elementi puntuali) - Documentazione QGIS (2025). Documentazione: [https://docs.qgis.org/3.40/it/docs/gentle\\_gis\\_introduction/vector\\_data.html#point-features-in-detail](https://docs.qgis.org/3.40/it/docs/gentle_gis_introduction/vector_data.html#point-features-in-detail), Consultato: settembre 2025.
- [87] QGIS Development Team. Modello Simple Feature - Documentazione QGIS (2025). Documentazione: [https://docs.qgis.org/3.40/it/docs/training\\_manual/spatial\\_databases/simple\\_feature\\_model.html#lesson-simple-feature-model](https://docs.qgis.org/3.40/it/docs/training_manual/spatial_databases/simple_feature_model.html#lesson-simple-feature-model), Consultato: settembre 2025.
- [88] TechPowerUp. AMD Ryzen 5 3600 Specs | TechPowerUp SSD Database (2025). Documentazione: <https://www.techpowerup.com/cpu-specs/ryzen-5-3600.c2132>, Consultato: settembre 2025.
- [89] TechPowerUp. Crucial MX500 1 TB Specs | TechPowerUp SSD Database (2025). Documentazione: <https://www.techpowerup.com/ssd-specs/crucial-mx500-1-tb.d77>, Consultato: settembre 2025.
- [90] TechPowerUp. Kingston A2000 1 TB Specs | TechPowerUp SSD Database (2025). Documentazione: <https://www.techpowerup.com/ssd-specs/kingston-a2000-1-tb.d285>, Consultato: settembre 2025.
- [91] J. Topf. Osmium Tool - A multipurpose command line tool based on the Osmium Library (2025). Documentazione: <https://osmcode.org/osmium-tool/>, Consultato: settembre 2025.
- [92] C. van der Wel and PyGEOS Developers. pygeos/pygeos: 0.14 Release (0.14) (2022). *Zenodo*, Consultato: settembre 2025. Documentazione: <https://pygeos.readthedocs.io/en/stable/>. doi: <https://doi.org/10.5281/zenodo.7426574>.
- [93] Brendan C. Ward and pyogrio Contributors. pyogrio - bulk-oriented spatial vector file I/O using GDAL/OGR (2025). Documentazione: <https://pyogrio.readthedocs.io/en/latest/>, Consultato: settembre 2025.
- [94] J. Whitaker and Open Source Contributors. Pyproj - Python interface to PROJ (2025). Documentazione: <https://pyproj4.github.io/pyproj/stable/>, Consultato: settembre 2025.
- [95] Wikipedia and Wikimedia Movement. CPU-bound - Wikipedia (2019). Pagina web: <https://it.wikipedia.org/wiki/CPU-bound>, Consultato: ottobre 2025.
- [96] Wikipedia and Wikimedia Movement. Metodo forza bruta - Wikipedia (2025). Pagina web: [https://it.wikipedia.org/wiki/Metodo\\_forza\\_bruta](https://it.wikipedia.org/wiki/Metodo_forza_bruta), Consultato: ottobre 2025.

- [97] Wikipedia and Wikimedia Movement. Throughput - Wikipedia (2024). Pagina web: <https://it.wikipedia.org/wiki/Throughput>, Consultato: ottobre 2025.
- [98] Wikipedia and Wikimedia Movement. Bounding box - Wikipedia (2024). Pagina web: [https://it.wikipedia.org/wiki/Bounding\\_box](https://it.wikipedia.org/wiki/Bounding_box), Consultato: settembre 2025.
- [99] Wikipedia and Wikimedia Movement. Caso d'uso (informatica) - Wikipedia (2020). Pagina web: [https://it.wikipedia.org/wiki/Caso\\_d%27uso\\_\(informatica\)](https://it.wikipedia.org/wiki/Caso_d%27uso_(informatica)), Consultato: settembre 2025.
- [100] Wikipedia and Wikimedia Movement. I/O bound - Wikipedia (2019). Pagina web: [https://it.wikipedia.org/wiki/I/O\\_bound](https://it.wikipedia.org/wiki/I/O_bound), Consultato: settembre 2025.
- [101] Wikipedia and Wikimedia Movement. Modello digitale di elevazione - Wikipedia (2025). Pagina web: [https://it.wikipedia.org/wiki/Modello\\_digitale\\_di\\_elevazione](https://it.wikipedia.org/wiki/Modello_digitale_di_elevazione), Consultato: settembre 2025.
- [102] Wikipedia and Wikimedia Movement. OLAP - Wikipedia (2024). Pagina web: <https://it.wikipedia.org/wiki/OLAP>, Consultato: settembre 2025.
- [103] Wikipedia and Wikimedia Movement. Pixel - Wikipedia (2025). Pagina web: <https://it.wikipedia.org/wiki/Pixel>, Consultato: settembre 2025.
- [104] Wikipedia and Wikimedia Movement. Point in polygon - Wikipedia (2025). Pagina web: [https://en.wikipedia.org/wiki/Point\\_in\\_polygon](https://en.wikipedia.org/wiki/Point_in_polygon), Consultato: settembre 2025.
- [105] Wikipedia and Wikimedia Movement. R-tree - Wikipedia (2025). Pagina web: <https://it.wikipedia.org/wiki/R-tree>, Consultato: settembre 2025.
- [106] Wikipedia and Wikimedia Movement. Sardinia satellite - Sardegna (isola) - Wikipedia (2011). Immagine web: [https://it.wikipedia.org/wiki/Sardegna\\_\(isola\)#/media/File:Sardinia\\_satellite.jpg](https://it.wikipedia.org/wiki/Sardegna_(isola)#/media/File:Sardinia_satellite.jpg), Consultato: settembre 2025.
- [107] Wikipedia and Wikimedia Movement. Simple Features - Wikipedia (2025). Pagina web: [https://en.wikipedia.org/wiki/Simple\\_Features](https://en.wikipedia.org/wiki/Simple_Features), Consultato: settembre 2025.
- [108] Wikipedia and Wikimedia Movement. Spatial reference system - Wikipedia (2025). Pagina web: [https://en.wikipedia.org/wiki/Spatial\\_reference\\_system](https://en.wikipedia.org/wiki/Spatial_reference_system), Consultato: settembre 2025.
- [109] X. Yao and G. Li. Big spatial vector data management: a review (2018). *Big Earth Data*, 2(1):108–129, Consultato: settembre 2025. Articolo: <https://www.tandfonline.com/doi/full/10.1080/20964471.2018.1432115>. doi: <https://doi.org/10.1080/20964471.2018.1432115>.